Formal Languages & Automata Theory

CG

November 7, 2026

Contents

1 Programming languages: an overview **2**

1.1 What is a programming language . . . 2
1.2 What is a program . . . 3
1.3 Languages seen in class . . . 4
1.4 What makes a good programming language . . . 4
1.5 A brief history . . . 6
1.6 Design goals in tension . . . 7
1.7 Paradigms . . . 7
1.8 Why formal definitions . . . 8
1.9 Exercises . . . 8

2 Grammars and language definitions **9**

2.1 Grammars . . . 9
2.2 Alphabets, words and languages . . . 9
2.3 BNF and EBNF . . . 10
2.4 Concrete syntax . . . 10
2.5 Abstract syntax . . . 11
2.6 A complete example: a spreadsheet language . . . 12
2.7 Derivations . . . 14
2.8 The Chomsky hierarchy . . . 15
2.9 Ambiguity . . . 15
2.10 Exercises . . . 16

3 Finite automata **17**

3.1 Labeled transition systems . . . 17
3.2 Deterministic finite automata . . . 18
3.3 Nondeterministic finite automata . . . 19

3.4 From NFA to DFA . . . 21
3.5 Combining automata . . . 23
3.6 Exercises . . . 26

4 Regular languages **29**
4.1 Regular grammars . . . 29
4.2 Regular expressions . . . 31
4.3 Recognizing regular languages . . . 32
4.4 A regular DSL: dates . . . 32
4.5 Languages that are not regular . . . 34
4.6 Regular expressions in practice . . . 35
4.7 Exercises . . . 36

5 Context-free languages **38**
5.1 Context-free grammars . . . 38
5.2 Parse trees . . . 40
5.3 Ambiguity . . . 41
5.4 Removing ambiguity . . . 42
5.5 Left recursion . . . 44
5.6 Pushdown automata . . . 45
5.7 Chomsky normal form . . . 48
5.8 A context-free DSL: LDBL . . . 48
5.9 Languages that are not context-free . . . 50
5.10 Closure properties . . . 51
5.11 Exercises . . . 51

1 Programming languages: an overview

1.1 What is a programming language

The course starts from a short definition.

Definition 1.1. A **programming language** is a system to write computer programs.

The definition is short on purpose, and the rest of the course fills it in. Figure 1 shows the parts that the first lecture adds to it. The syntax of a language says how programs look: which symbols can be used and how they combine into structures. The semantics says how programs work: what type each expression has, which environment gives values to the names, how names are bound, and how the program is represented internally when it runs. Around both sits the tooling (editors, compilers, debuggers), which does not change what a program means but decides how pleasant the language is to use.

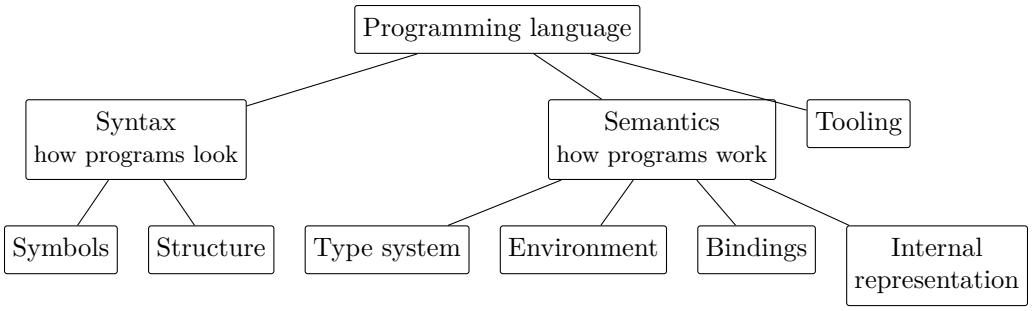


Figure 1: The components of a programming language, as presented in the first lecture.

Each block of these notes takes one of the boxes. Symbols and structure are the subject of this block and the next one (grammars, automata, lexing, parsing). Type systems, environments and bindings come after that, and internal representation is the subject of the runtime block.

1.2 What is a program

Take this text from the lecture:

```
fun arithmetic_logic(x,y) {
  a = 3+foo;
  x and y;
}
```

Whether it is a program depends on the language. A string is a program of a language L only if it follows the syntax of L, and it is a meaningful program only if it also follows the semantic rules of L. Without fixing L the question has no answer.

In the lecture the text is given to a parser for a small language whose statements are assignments and expressions. Figure 2 shows the tree the parser builds. Function definition, parameters and the second statement `x and y;` all fit the grammar. The first statement does not: the grammar of that language has no `+` operator, so after `a = 3` the parser finds `+foo` where it expects `;`. It reports the error at `foo` (dashed box in the figure), skips the offending tokens and continues. So the text is not a program of that language, even though every token in it is valid.

Passing the parser is not enough either. If the grammar had `+`, the statement `a = 3+foo` would still need `foo` to be bound to something, and `x and y` would need `x` and `y` to be booleans. Those checks belong to the semantics, and different languages make them at different moments.

The lecture shows this with a single line typed into seven interactive interpreters (Table 1). The text `a = b + c` is the same in all of them, and none of them runs it.

The differences are not in the text but in the rules each language applies to it. JavaScript, Ruby and Python look up `b` when the line is executed and fail there. Elixir, Haskell and C++ check names before the program runs. C++ complains about `a` first because it requires every variable to be declared before its first use, including the one being assigned. Prolog does not read `=` as assignment at all: it reads `a = b + c` as the question "does the atom `a` unify with the term `b + c`?", and the answer is no.

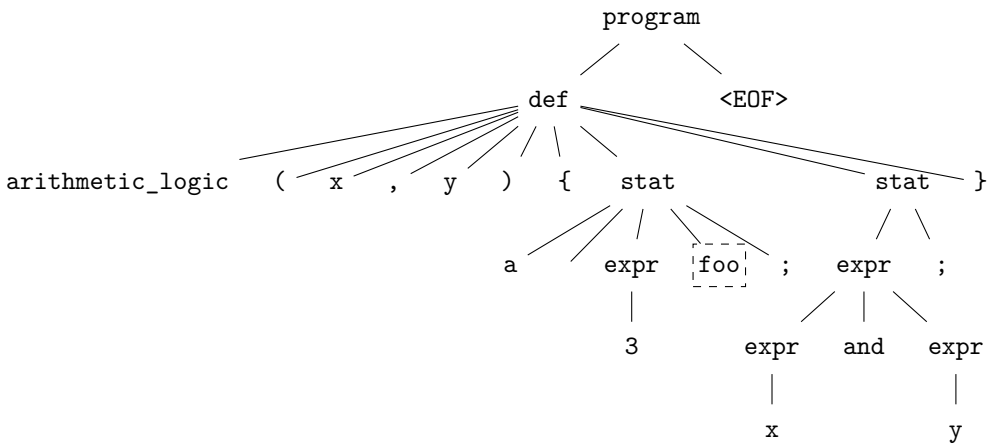


Figure 2: Parse tree for `arithmetic_logic`. The parser cannot place `foo` and marks it as an error.

Language	Response to `a = b + c`	When the problem is found
JavaScript	`ReferenceError: b is not defined`	while running
Ruby	`NameError: undefined local variable or method ‘b’`	while running
Python	`NameError: name ’b’ is not defined`	while running
Elixir	`undefined variable "b"`, `CompileError`	before running
Haskell	`Variable not in scope: b`, and the same for `c`	before running
C++	`Use of undeclared identifier ’a’`	before running
Prolog	`false.`	while running

Table 1: The same line in seven languages. Each one reads it with its own semantics.

1.3 Languages seen in class

In the first lecture each student writes a function that computes the factorial of a number, and then the same function is shown in several languages. Table 2 summarizes them. The point is that the same algorithm can look very different depending on what the language was designed for.

For reference, here is the factorial in the two languages used for code in these notes. In Scheme:

```
(define (factorial n)
  (if (< n 2)
      1
      (* n (factorial (- n 1)))))
```

and in Rascal:

```
int factorial(int n) = n < 2 ? 1 : n * factorial(n - 1);
```

Both are recursive definitions by cases, and both files include tests.

1.4 What makes a good programming language

Pratt and Zelkowitz [4] give eight criteria, and the lecture follows them.

Language	What it shows
Ada	Structured, statically typed, imperative and object-oriented. Strong typing, design by contract and concurrency are built in; the compiler is expected to catch errors before the program runs. The factorial uses subtypes `Positive` and `Natural`, so a negative argument is a type error.
Prolog	Logic programming. A program is a set of facts and rules that define relations, and computation starts with a query. The factorial is two clauses relating `X` and `F`.
Scratch	Block-based visual language for children aged 8 to 16. Programs are built by snapping blocks together, so syntax errors cannot be written.
ASCII Dots	Esoteric language based on ASCII art. Dots travel along paths drawn with characters and are transformed by the operators they cross.
CLU	Introduced abstract data types, call by sharing, iterators, multiple return values and type-safe parameterized types. Its factorial `signals negative` instead of returning a wrong value.
UML	General-purpose visual modeling language with structure, behavior and interaction diagrams.
Action!	Procedural language for Atari 8-bit computers, similar to ALGOL, compiled to fast 6502 code. Its factorial reports errors through a pointer argument because the result type cannot hold them.
0815	Esoteric language based on a queue; the factorial is written with jumps and labels such as `}:f:` and `^:f:`.
Piet	Stack-based esoteric language whose programs are images. It uses 20 colors; transitions between colors encode the operations.
SCuLPT	Simple Cubic Language for Programming Tasks. Programs are built from tangible 3D pieces instead of typed text, and the language is Turing complete.

Table 2: Languages used in the first lecture.

Clarity, simplicity and unity of concepts. A language is a tool for thinking about algorithms, writing them down and communicating them. It should offer a small set of concepts, with simple and regular rules to combine them, and it should be unambiguous. The dangling `else` of Section 5 is an example of a construct where the grammar alone does not say what a program means.

Clarity of program structure. The syntax should not mislead, and the structure of a program should reflect the logical structure of the algorithm.

Naturalness for the application. The language should provide the data structures, operations, control structures and syntax that the problem needs. The date language of Section 4 is natural for dates and useless for anything else, which is what a domain-specific language aims for.

Ease of extension. Extensions should use simple mechanisms built from primitive features of the language, so that they feel like part of it.

External support. Tools for testing, editing and storing programs, and good documentation, decide much of a language's success.

Ease of verification. Programs can be proven correct with a formal method, validated by tests that compare outputs against the specification, or desk-checked by reading them. In all three cases, simple syntax and semantics make the work easier.

Portability. Programs should move between computing environments without changes.

Efficiency and cost. There are several costs: executing the program (compiler optimizations and runtime support), translating it (fast compilation matters for programs compiled often and run rarely), creating it (design, coding, testing, modifying) and maintaining it.

MacLennan [3] gives a longer list of design principles, shown in Table 3. Several of them come back later in the course: orthogonality and regularity when we compare grammars, information hiding when we discuss scopes, security when we discuss type systems.

Principle	Meaning
Abstraction	Do not require something to be stated more than once; factor out recurring patterns.
Automation	Automate mechanical, tedious or error-prone activities.
Defense in depth	Have a series of defenses, so an error missed by one is caught by another.
Information hiding	Users of a module get all the information they need and nothing more; implementers get all they need to implement it.
Labeling	Do not require the user to know absolute positions in a list; give items meaningful labels.
Localized cost	Users pay only for what they use.
Manifest interface	All interfaces are apparent in the syntax.
Orthogonality	Independent functions are controlled by independent mechanisms.
Portability	Avoid features that depend on a particular machine.
Preservation of information	Let the user state what they know and the compiler might need.
Regularity	Regular rules without exceptions are easier to learn, use, describe and implement.
Security	No program that violates the definition of the language should escape detection.
Simplicity	A minimum number of concepts with simple rules for combining them.
Structure	The static structure of the program corresponds to the dynamic structure of the computation.
Syntactic consistency	Similar things look similar; different things look different.
Zero-one-infinity	The only reasonable numbers are zero, one and infinity.

Table 3: MacLennan's principles of language design.

1.5 A brief history

The history of programming languages explains why some ideas exist. Three periods are enough for this course.

The first languages were machine code and assembly (1940s and 1950s). A program was a sequence of numeric opcodes, later written with mnemonic names. One line was one machine instruction, and a program only ran on the processor it was written for.

The early high-level languages came next (1950s and 1960s). FORTRAN (1957), Lisp (1958) and COBOL (1959) separated source code from the hardware. FORTRAN gave scientists expressions

and arrays instead of registers. Lisp introduced code as data, recursion as the main control structure and garbage collection.

From the 1970s on, Pascal and C brought structured control flow without `goto`, Smalltalk and then C++ and Java brought objects, and ML and Haskell brought typed functional programming with type inference. Each step gave up some low-level control in exchange for a guarantee or for less work for the programmer. Types and semantics, which come later in the course, are the tools that state precisely what was gained.

1.6 Design goals in tension

The criteria of the previous section pull in different directions, and every language resolves the conflicts in its own way.

Expressiveness and simplicity. A more expressive language lets programmers say more, often in fewer lines, but a larger language is harder to learn, to implement and to reason about. This is the tension between the first criterion (simplicity) and the third (naturalness for the application).

Safety and flexibility. A safe language rejects some programs on purpose. A type checker that rejects a program before it runs removes a class of runtime errors, and it also rejects some programs that would have run correctly. Table 1 shows both choices: Haskell rejects `a = b + c` before running, Python runs it and fails.

Performance and abstraction. Garbage collection, dynamic dispatch and generic containers make programs easier to write and reuse, and they cost time and memory. A Python list of integers holds pointers to boxed objects in the heap; a C array of `int` is a contiguous block of machine integers. The runtime block of the course returns to this difference.

Readability and writability. A notation that is quick to write can be slow to read. Verbose languages are slow to write and easy to read cold. Most general-purpose languages lean toward readability, since code is read more often than it is written. Domain-specific languages sometimes go the other way, because their users already know the notation of the domain.

1.7 Paradigms

A **paradigm** is a way of thinking about computation: what a step is, what state means, and how control flow is expressed. Most languages mix paradigms.

In the imperative paradigm a program is a sequence of statements that change the state; the basic unit is the assignment and control flow is explicit. In the functional paradigm a program is an expression built from function applications, and computation is evaluation; in the pure case there is no mutable state. In the logic paradigm a program is a set of facts and rules and running it means asking a query; Prolog in Table 2 is the example. In the object-oriented paradigm a program is a set of objects that group state and behavior and communicate by calling methods.

The same small problem, adding the even numbers of a list, looks different in each style. Imperative, in Python:

```
def sum_evens(xs):
    total = 0
    for x in xs:
        if x % 2 == 0:
            total += x
    return total
```

The accumulator and the loop are explicit. Functional, in Haskell:

```
sumEvens :: [Int] -> Int
sumEvens = sum . filter even
```

There is no accumulator and no loop: `filter even` selects the elements and `sum` adds them, and the two functions are composed. Declarative, in SQL:

```
SELECT SUM(value) FROM numbers WHERE value % 2 = 0;
```

The query states the result and leaves the strategy to the database engine. A logic version in Prolog would be three clauses: the sum of the empty list is 0, an even head is added to the sum of the tail, and an odd head is skipped.

1.8 Why formal definitions

Everything in this section so far was argued in prose. Prose is enough to start, but it does not settle disagreements. Whether `arithmetic_logic` is a program, or whether `a = b + c` is an error, can only be decided with a precise definition of the language. The course builds that definition in layers. Grammars say which strings are programs (Section 2). Automata give a model of computation that recognizes them and let us prove facts such as "no finite automaton recognizes balanced parentheses" (Sections 3 and 4). Type systems and semantics, later in the course, state what programs mean and which ones are rejected.

1.9 Exercises

Exercise 1.2. Write a function that computes the factorial of a number in a language you know. Then answer: what does your function do with a negative argument, and how would Ada, CLU and Action! handle that case according to Table 2?

Solution. The recursive definition used in these notes returns 1 for every $n < 2$, so it returns 1 for negative numbers, which is not a factorial. Ada declares the parameter as `Positive`, so a negative argument is a type error. CLU declares `signals (negative)` and raises that signal. Action! writes an error code through the pointer `err` (1 for negative, 2 for a result that does not fit) and returns 0.

Exercise 1.3. Is `arithmetic_logic` a program? Answer for the grammar of Figure 2, and then for a grammar that also has `+`. In the second case, what else would the language need to check?

Solution. For the grammar of the figure it is not a program: the first statement cannot be derived because there is no rule for `+`. With `+` in the grammar the text is syntactically a program. The language would still need to check that `foo` is bound (a scope rule), that `3+foo` adds two numbers

and that `x and y` combines two booleans (type rules). Whether `a` needs a declaration depends on the language, as the C++ row of Table 1 shows.

Exercise 1.4. For each language in Table 1, say whether the problem with `a = b + c` is found by the syntax or by the semantics, and explain why Prolog answers `false` instead of reporting an error.

Solution. In all seven languages the line is syntactically valid; every response comes from the semantics. Six languages complain about a name with no binding, at run time or before it. Prolog reads `=` as unification. The left side is the atom `a` and the right side is the compound term `+(b, c)`; an atom unifies only with itself or with a variable, so unification fails and the query answers `false`.

Exercise 1.5. Choose two of Pratt and Zelkowitz's criteria and one language from Table 2. Explain how the language satisfies or fails each criterion, with a concrete example.

Exercise 1.6. Pick two languages you know. For each of the four tensions of this section, say which way each language leans, with an example from each.

2 Grammars and language definitions

A programming language is defined by its syntax and its semantics. This section is about the syntactic definition, which is given by a grammar. The presentation follows Lee [2, Ch. 2].

2.1 Grammars

Definition 2.1. A **grammar** is a 4-tuple $\mathcal{G} = \langle N, \Sigma, S_0, P \rangle$ where N is a finite set of **nonterminal** symbols (the syntactic categories), Σ is a finite set of **terminal** symbols (the alphabet the syntax is made of), disjoint from N, $S_0 \in N$ is the **initial** symbol, and $P \subseteq (N \cup \Sigma)^* \times (N \cup \Sigma)^*$ is a finite set of **production rules**, written $\alpha \to \beta$, where α contains at least one nonterminal.

The production rules define how the words of the language are formed. Most grammars in this course have a single nonterminal on the left of every rule; those are the context-free grammars of Section 5. The general form is stated here so that the Chomsky hierarchy at the end of the section makes sense.

2.2 Alphabets, words and languages

Definition 2.2. An **alphabet** Σ is a finite, nonempty set of symbols. A **word** of length m over Σ is a sequence of m symbols of Σ, written $\omega = \sigma_1 \sigma_2 \cdots \sigma_m$, with $|\omega| = m$. The **empty word**, of length 0, is written λ (or ε). Σ^* is the set of all words over Σ, including λ.

The lecture uses three alphabets: the Spanish letters $\{a, b, \ldots, z\}$, the digits $\{0, 1, \ldots, 9\}$ and a set of emoji. `hola` is a word of length 4 over the first one, and `iokdafpodsaf` is a word of length 12 over the same alphabet even though it means nothing in Spanish. A word only needs its symbols to come from the alphabet. Whether it belongs to a language is a separate question.

Definition 2.3. A **language** over Σ is a set of words $L \subseteq \Sigma^*$. When L is given by a grammar, it contains the words that the production rules can generate.

Two examples from the lecture. The language of even binary numbers is

$$L_{even} = \{\sigma_1 \cdots \sigma_m \in \{0,1\}^* \mid m \geq 1,\ \sigma_m = 0\},$$

the binary words that end in 0. The language $\{\lambda\}$ has exactly one word, the empty one. It is different from the empty language $\emptyset$, which has no words at all. The empty word belongs to Σ^* for every alphabet, but a particular language may or may not contain it: $\lambda \in \{\lambda\}$, while $\lambda \notin L_{even}$.

Operations on languages. Since languages are sets, union, intersection and complement apply directly. Two more operations come from words. The **concatenation** of L_1 and L_2 is $L_1L_2 = \{xy \mid x \in L_1,\ y \in L_2\}$. The **Kleene star** of L is $L^* = \bigcup_{i \geq 0} L^i$, where $L^0 = \{\lambda\}$ and $L^{i+1} = LL^i$. With $L_1 = \{a, ab\}$ and $L_2 = \{b, \lambda\}$ we get $L_1L_2 = \{ab, a, abb\}$ (the word ab appears twice and counts once), and L_1^* contains $\lambda, a, ab, aa, aab, aba, \ldots$. Union, concatenation and star are the three operations behind regular expressions (Section 4).

2.3 BNF and EBNF

Writing grammars as 4-tuples is not practical. John Backus and Peter Naur defined a notation for the production rules when they specified ALGOL 60, and it is still the standard way to write grammars. In **BNF**:

- production rules are written with `::=`;
- terminal symbols are written between quotes;
- nonterminals are written without quotes;
- rules for the same nonterminal are grouped with `|`.

For example, `S ::= "a" S "b" | "ab"` generates ab, $aabb$, $aaabbb$ and so on. The extended form, **EBNF**, adds the operators of regular expressions: parentheses group symbols, `*` means zero or more repetitions of the previous item, `+` one or more, and `?` makes it optional.

EBNF does not generate more languages than BNF. Every repetition can be replaced by a new nonterminal with a recursive rule. In the lecture this is shown with the grammar of BNF written in BNF: rules such as `opt-whitespace ::= " " opt-whitespace | ""` use recursion where EBNF would write `" "*`.

2.4 Concrete syntax

A grammar defines the **concrete syntax** of a language: the exact form programs take when written. Ada's factorial (Section 1), a UML class diagram and a Scratch program have very different concrete syntax.

The lecture asks for a BNF definition of lists from this description: the empty list `[]` is a list, and a number followed by `|` and a list is a list. The answer is

```
LIST   ::= "[" "]" | NUMBER "|" LIST
NUMBER ::= ("0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9")+
```

Word	In `LIST`?	Reason
`3 | 14 | []`	yes	two numbers, then the empty list
`[]`	yes	the base rule
`[-7, 3, 14]`	no	`-` and `,` are not terminals of the grammar
`[1 | "hello" | 3.14 | True]`	no	`"hello"`, `3.14` and `True` are not numbers
`1 | [] | 7 | 9 | 3 | []`	no	after `[]` nothing else can follow
`1 | [] | [7 | 9 | 3 ]`	no	same, and `[7 | 9 | 3 ]` does not end in `[]`

Table 4: The list examples of the lecture.

Table 4 checks the candidates from the lecture against the grammar. Only the first two can be derived. A list always ends with `[]`, the only thing to the left of a `|` is a number, and the grammar has no brackets around elements, no commas, no signs and no strings.

The same grammar in Rascal, where `lexical` defines the shape of numbers, `layout` says that spaces may appear between symbols, and each alternative gets a label:

```
lexical NUMBER = [0-9]+ !>> [0-9];
layout Spaces = [\ ]* !>> [\ ];

start syntax LIST
  = empty: "[" "]"
  | cons: NUMBER "|" LIST
  ;
```

The module `Lists.rsc` has one test per row of Table 4.

2.5 Abstract syntax

The **abstract syntax** of a language is a high-level representation of its programs that keeps the structure and drops the notation. It does not depend on the concrete syntax: the same abstract syntax could be written with brackets, with indentation or with blocks. Figure 3 shows the difference for the list `3 | 14 | []`. The parse tree has a leaf for every symbol of the text, including `|`, `[` and `]`. The abstract syntax tree (AST) only keeps what matters: a list built from 3, then 14, then the empty list.

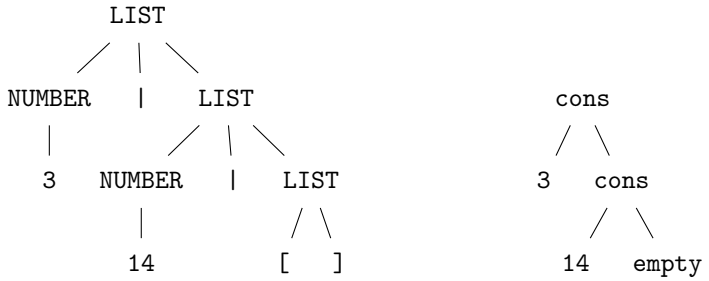


Figure 3: Parse tree (left) and abstract syntax tree (right) of `3 | 14 | []`.

Algebraic data types are the usual way to define an abstract syntax. A type is defined by its constructors, and each constructor says which parts a node has. In Rascal:

```
data List
  = empty()
  | cons(int head, List tail)
  ;

List toAst(str text) = unsetRec(implode(#List, parse(#start[LIST], text).top));
```

`implode` turns a parse tree into a value of the data type, matching each labeled alternative of the grammar with the constructor of the same name. For `3 | 14 | []` it returns `cons(3, cons(14, empty()))`.

The lecture gives the abstract syntax of a small expression language with this grammar:

```
expr    ::= literal | unary | binary
unary   ::= "¬" expr
binary  ::= expr "*" expr | expr "/" expr | expr "+" expr | expr "-" expr
literal ::= "0" .. "9" | "true" | "false"
```

and this data type:

```
data Expr
  = integer(int intValue)
  | \true()
  | \false()
  | not(Expr arg)
  | mul(Expr lhs, Expr rhs)
  | div(Expr lhs, Expr rhs)
  | add(Expr lhs, Expr rhs)
  | sub(Expr lhs, Expr rhs)
  ;
```

(The slide declares the right operand as `Express rhs`; the type is `Expr`. It also lists a constructor `var(Id id)`, which has no rule in the grammar above.)

Of the three inputs `3 * 4`, `¬ false + true` and `1 + 1 + 1`, all three follow the grammar. The second is not a well-formed program: it adds a boolean (`¬ false`) to another boolean. The grammar cannot see this, because `expr` does not distinguish numbers from booleans. A function over the abstract syntax can:

```
str kind(integer(_)) = "int";
str kind(\true())    = "bool";
str kind(\false())   = "bool";
str kind(not(e))     = kind(e) == "bool" ? "bool" : "error";
default str kind(Expr e) = arith(e.lhs, e.rhs);

str arith(Expr l, Expr r) = kind(l) == "int" && kind(r) == "int" ? "int" : "error";
```

This function is a first, very small type checker. Type systems are the subject of a later block.

2.6 A complete example: a spreadsheet language

The lecture builds a grammar for spreadsheet formulas from this description. A spreadsheet is made of cells, and a cell contains a value, a reference to another cell or a formula. In A1-style addressing a cell is referenced by column and row, as in `D7`. A reference is relative or absolute; a `$` in front of

a part makes that part absolute, so `$D7` fixes the column and leaves the row relative. Functions operate on groups of cells given as ranges: `SUM(A1:A5)` adds `A1` through `A5`.

The first step is to collect the symbols. From inputs such as `=SUM(A1:A5)`, `=B7 + 3` and `=AVERAGE(AB$13, 9, 3.14, Z1480)`:

$$\Sigma = \{\texttt{=}, \texttt{\$}, \texttt{*}, \texttt{+}, \texttt{/}, \texttt{(}, \texttt{)}, \texttt{-}, \texttt{:}, \texttt{,}, \texttt{.}, \texttt{SUM}, \texttt{AVERAGE}, \texttt{MOD}, \texttt{0..9}, \texttt{A..Z}\}.$$

The second step is to name the categories of elements: values, formulas (simple, compound, enumerated, range), references (relative and fixed), functions, operations, digits and letters. These become the nonterminals. The third step is the production rules:

```
ss                 ::= value | formula
formula            ::= "=" internal-formula
internal-formula   ::= value | ref | simple-formula
                     | compound-formula | enumerated-formula
simple-formula     ::= internal-formula operation internal-formula
compound-formula   ::= enumerated-formula | function "(" ref ":" ref ")"
enumerated-formula ::= function "(" (ref | value) ("," (ref | value))* ")"
ref                ::= "$"? letter letter? "$"? digit digit?
operation          ::= "+" | "-" | "*" | "/"
function           ::= "MOD" | "SUM" | "AVERAGE"
value              ::= digit+
letter             ::= "A" .. "Z"
digit              ::= "0" .. "9"
```

The slides use `operation` without giving its rule; the rule above takes the operators from Σ.

Here is a derivation of `=B7 + 3`. Each step replaces one nonterminal with the right side of one of its rules:

$$\begin{aligned}
ss &\Rightarrow \textit{formula} \\
&\Rightarrow \texttt{=}\ \textit{internal} \\
&\Rightarrow \texttt{=}\ \textit{simple} \\
&\Rightarrow \texttt{=}\ \textit{internal operation internal} \\
&\Rightarrow \texttt{=}\ \textit{ref operation internal} \\
&\Rightarrow \texttt{=}\ \textit{ref} \texttt{ +}\ \textit{internal} \\
&\Rightarrow \texttt{=}\ \textit{ref} \texttt{ +}\ \textit{value} \\
&\Rightarrow \texttt{=}\ \textit{letter digit} \texttt{ +}\ \textit{value} \\
&\Rightarrow \texttt{=}\ \texttt{B}\ \textit{digit} \texttt{ +}\ \textit{value} \\
&\Rightarrow \texttt{=}\ \texttt{B7} \texttt{ +}\ \textit{value} \\
&\Rightarrow \texttt{=}\ \texttt{B7} \texttt{ +}\ \textit{digit} \\
&\Rightarrow \texttt{=}\ \texttt{B7} \texttt{ +}\ \texttt{3}
\end{aligned}$$

(*internal* and *simple* abbreviate *internal-formula* and *simple-formula*.) Figure 4 shows the same derivation as a tree.

The grammar in Rascal follows the BNF almost line by line. Letters, digits, values and references are `lexical` because no spaces are allowed inside them:

```
layout Spaces = [\ ]* !>> [\ ];

lexical Digit  = [0-9];
lexical Letter = [A-Z];
lexical Value  = Digit+ !>> [0-9];
lexical Ref    = "$"? Letter Letter? "$"? Digit Digit? !>> [0-9];
```

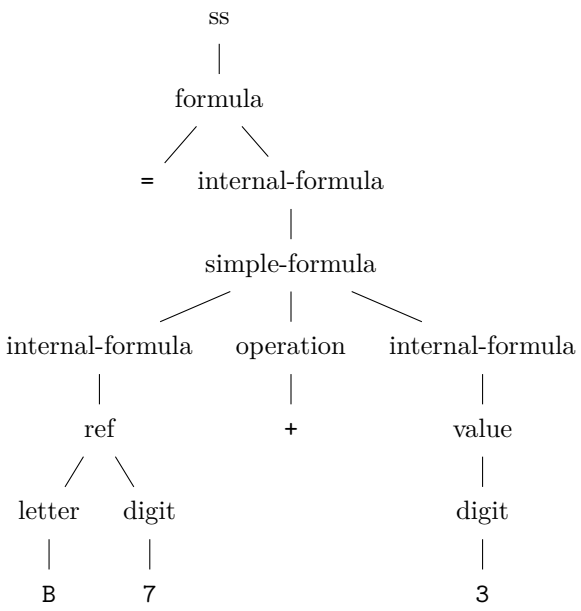


Figure 4: Derivation tree of `=B7 + 3` in the spreadsheet grammar.

```
lexical Function = "MOD" | "SUM" | "AVERAGE";

syntax Operation = "+" | "-" | "*" | "/";

start syntax SS = Value | Formula;

syntax Formula = "=" Internal;

syntax Internal
  = Value
  | Ref
  | SimpleFormula
  | CompoundFormula
  | EnumeratedFormula
  ;

syntax SimpleFormula     = Internal Operation Internal;
syntax CompoundFormula   = EnumeratedFormula | Function "(" Ref ":" Ref ")";
syntax EnumeratedFormula = Function "(" Arg ("," Arg)* ")";
syntax Arg = Ref | Value;
```

The tests in `Spreadsheet.rsc` check every input of the lecture, including the ones in the exercises below.

2.7 Derivations

A grammar generates words by derivation. Start from S_0 and repeatedly replace a nonterminal with the right side of one of its rules, until only terminals remain.

Definition 2.4. If $\alpha \to \beta \in P$, then $\gamma\alpha\delta \Rightarrow \gamma\beta\delta$ (one derivation step). $\Rightarrow^*$ is zero or more steps. A **sentential form** is any string reachable from S_0, and the language generated by $\mathcal{G}$ is $L(\mathcal{G}) = \{\omega \in \Sigma^* \mid S_0 \Rightarrow^* \omega\}$.

When a sentential form has several nonterminals there is a choice of which one to replace. A **leftmost derivation** always replaces the leftmost nonterminal and a **rightmost derivation** the rightmost one. The derivation of `=B7 + 3` above is leftmost. Both orders build the same tree; they matter for parsing, where top-down parsers follow leftmost derivations and bottom-up parsers follow rightmost derivations in reverse (Section 5 and the next block).

2.8 The Chomsky hierarchy

Grammars are classified by the shape of their rules (Table 5). Each class is strictly contained in the next one (Figure 5), and each corresponds to a kind of machine that recognizes it.

Type	Rules	Recognized by
3, regular	$A \to a$, $A \to aB$ (or $A \to Ba$)	finite automata (Section 3)
2, context-free	$A \to \beta$	pushdown automata (Section 5)
1, context-sensitive	$\alpha A\beta \to \alpha\gamma\beta$, $\gamma \neq \lambda$	linear-bounded automata
0, unrestricted	$\alpha \to \beta$	Turing machines

Table 5: The Chomsky hierarchy. $A, B \in N$, $a \in \Sigma$, $\alpha, \beta, \gamma \in (N \cup \Sigma)^*$.

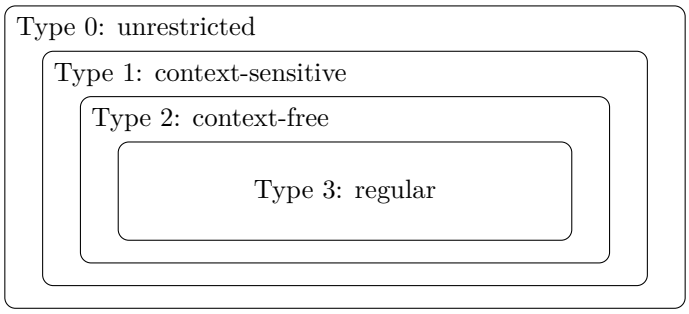


Figure 5: Each class of languages contains the one below it.

The list grammar and the spreadsheet grammar are context-free. Most of the syntax of a programming language is context-free. Rules such as "a variable must be declared before it is used" are not, and in practice they are checked after parsing, in semantic analysis, instead of with a context-sensitive grammar.

2.9 Ambiguity

Definition 2.5. A grammar is **ambiguous** if some word of its language has two different derivation trees.

The spreadsheet grammar is ambiguous. In `=1 + 2 * 3`, the rule

```
simple-formula ::= internal-formula operation internal-formula
```

can split the formula at the `+` or at the `*` (Figure 6). The two trees group the operations differently, and if the formula were evaluated they would give 7 and 9. The Rascal tests confirm that the parser finds both trees. Section 5 shows how to rewrite a grammar so that precedence and associativity are decided by the rules.

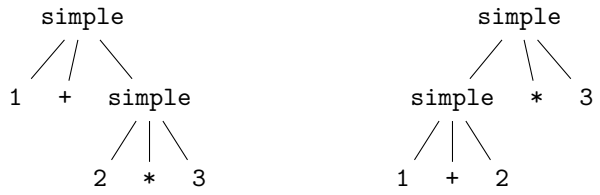


Figure 6: Two trees for `=1 + 2 * 3` (only the `simple-formula` nodes are shown).

2.10 Exercises

Exercise 2.6. For each word, decide whether the spreadsheet grammar generates it. If it does not, find where the derivation gets stuck. `=MOD(M9,L)`, `A5`, `=A3 ++ 7`, `=AVERAGE(A1:A5)SUM(A1:A5)`, `=SUM(A1,A2,)`, `=3A + B5`.

Solution. None of them is in the language.

- `=MOD(M9,L)`. The lecture first tries `compound-formula`:

  $$\ldots \Rightarrow \texttt{= function(ref:ref)} \Rightarrow \texttt{= MOD(ref:ref)} \Rightarrow \texttt{= MOD(M9:ref)}.$$

  The next symbol of the word is `,`, not `:`, so this path fails. With `enumerated-formula` the first argument works, but the second, `L`, is neither a `ref` (a reference needs at least one digit) nor a `value`.

- `A5`. Without `=` it can only be a `value`, and a value is made of digits.

- `=A3 ++ 7`. `simple-formula` puts exactly one `operation` between two formulas, and `+7` is not a formula.

- `=AVERAGE(A1:A5)SUM(A1:A5)`. Two formulas next to each other need an `operation` between them.

- `=SUM(A1,A2,)`. In `enumerated-formula` every comma must be followed by a reference or a value.

- `=3A + B5`. `3A` is not a value (it has a letter) and not a reference (a reference starts with a letter).

Exercise 2.7. Which of the inputs on the "Language symbols" slide are in the language? `=SUM(A1:A5)`, `=B7 + 3`, `=AVERAGE(AB$13, 9, 3.14, Z1480)`, `9`, `=mod(M9,L8)`, `=SUM(A1;A5) + 3`.

Solution. `=SUM(A1:A5)`, `=B7 + 3` and `9` are. The `AVERAGE` formula is not: `value ::= digit+` has no decimal point, so `3.14` cannot be derived, and `ref` allows at most two digits, so `Z1480` cannot either. The symbol `.` is in Σ, but no rule uses it. To accept the formula, `value` would need an optional fractional part and `ref` more digits. `=mod(M9,L8)` fails because function names are uppercase, and `=SUM(A1;A5) + 3` because `;` is not in Σ.

Exercise 2.8. Write a BNF grammar for lists whose elements may be numbers or lists, so that `1 | [] | [7 | 9 | 3 | []]` is a list. Give the derivation tree of `1 | [] | []`.

Solution. An element can be a number or a list in brackets:

```
LIST    ::= "[" "]" | ELEMENT "|" LIST
ELEMENT ::= NUMBER | "[" LIST "]" | "[" "]"
```

The alternative `"[" "]"` in `ELEMENT` is the empty list used as an element. In `1 | [] | []` the root is `LIST` with children `ELEMENT` (deriving `1`), `|` and `LIST`; that `LIST` has children `ELEMENT` (deriving `[]`), `|` and `LIST` (deriving the final `[]`).

Exercise 2.9. Give a grammar for $\{a^n b^n \mid n \geq 0\}$ and a leftmost derivation of $aabb$.

Solution. `S ::= "a" S "b" | ""`. Then $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb$. The lecture's rule `S ::= "a" S "b" | "ab"` generates the same language without the empty word.

Exercise 2.10. Let $L_1 = \{\lambda, a\}$ and $L_2 = \{b, bb\}$. Write $L_1 L_2$ and the words of L_1^* of length at most 3.

Solution. $L_1 L_2 = \{b, bb, ab, abb\}$. Every word of L_1^* is a sequence of a's, so the words of length at most 3 are λ, a, aa, aaa.

3 Finite automata

A grammar generates the words of a language. An automaton goes the other way: it reads a word and decides whether it belongs to the language. This section follows Hopcroft, Motwani and Ullman [1, Ch. 2]. The lecture writes automata in the Gold language; here they are drawn as diagrams and, when they need to run, written in Rascal.

3.1 Labeled transition systems

Definition 3.1. A **labeled transition system** (LTS) is a triple $\mathcal{M} = \langle Q, \Sigma, \delta \rangle$ where Q is the set of states of the system, Σ is the set of actions (labels) and $\delta \subseteq Q \times \Sigma \times Q$ is the set of transitions. A transition $\langle q, \sigma, q' \rangle$ says that the system can move from q to q' when the action σ happens.

An automaton is an LTS with a starting point and a notion of success. The lecture motivates it with a purchase. A customer pays the store; the store ships the goods; the store redeems the money at the bank; the bank transfers the money to the store. Shipping can happen before or after the money is redeemed or transferred, but the purchase is complete only when the goods have been shipped and the money transferred. Figure 7 models this.

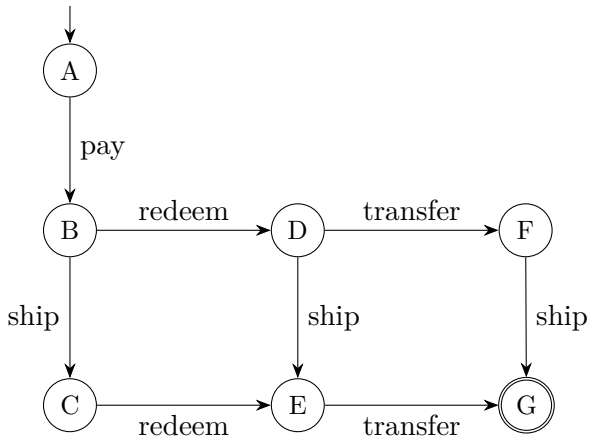


Figure 7: The purchase automaton. $\Sigma = \{\textit{pay}, \textit{ship}, \textit{redeem}, \textit{transfer}\}$, the initial state is A and the only final state is G.

Every path from A to G is a correct execution of the purchase. The word *pay ship redeem transfer* reaches G, and so does *pay redeem transfer ship*. The word *pay ship ship* does not: the goods cannot be shipped twice, so C has no transition for *ship*. In this example the symbols of the alphabet are whole actions, not characters.

3.2 Deterministic finite automata

Definition 3.2. A **deterministic finite automaton** (DFA) is a 5-tuple $\mathcal{A} = \langle Q, \Sigma, q_I, F, \delta \rangle$ where Q is a finite set of states, Σ is the input alphabet, $q_I \in Q$ is the initial state, $F \subseteq Q$ is the set of final states and $\delta : Q \times \Sigma \to Q$ is the transition function.

The automaton reads the word from an input tape, one symbol at a time, and each symbol moves it to the next state.

Definition 3.3. A word $\omega = \sigma_1 \cdots \sigma_n \in \Sigma^*$ is **accepted** by $\mathcal{A}$ if there are states $q_0, \ldots, q_n$ such that $q_0 = q_I$, $q_n \in F$ and $\delta(q_{i-1}, \sigma_i) = q_i$ for every $i \in \{1, \ldots, n\}$.

The same idea can be packed into one function that processes a whole word. The **extended transition function** $\hat{\delta} : Q \times \Sigma^* \to Q$ is defined by induction on the word:

$$\hat{\delta}(q, \lambda) = q \qquad\qquad \hat{\delta}(q, \omega\sigma) = \delta(\hat{\delta}(q, \omega), \sigma).$$

The empty word leaves the automaton where it is, and a word that ends in σ is processed by first processing the prefix and then taking one more step.

Definition 3.4. The **language of a DFA** is $L(\mathcal{A}) = \{\omega \in \Sigma^* \mid \hat{\delta}(q_I, \omega) \in F\}$.

Example: words that contain 01. The lecture builds a DFA over $\{0, 1\}$ that accepts the words containing the substring 01 (Figure 8). The state names say what the automaton has seen. In *Just1* it has seen only 1s (or nothing). A 0 moves it to *Read0*, where further 0s change nothing because the last symbol is still 0. A 1 after that completes the pattern, and from *Read01* every symbol keeps the word accepted.

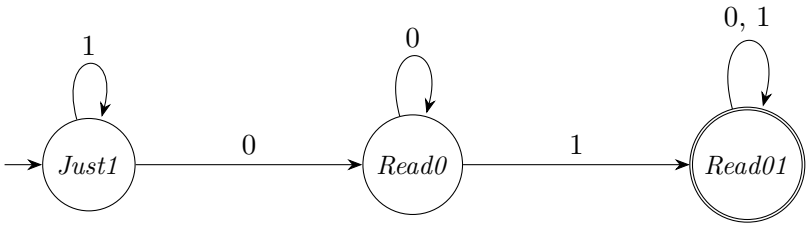


Figure 8: DFA for the words over $\{0, 1\}$ that contain 01.

For the word 1010: $\hat{\delta}(\mathit{Just1}, 1010) = \delta(\delta(\delta(\delta(\mathit{Just1}, 1), 0), 1), 0) = \delta(\delta(\delta(\mathit{Just1}, 0), 1), 0) = \delta(\delta(\mathit{Read0}, 1), 0) = \delta(\mathit{Read01}, 0) = \mathit{Read01}$, so 1010 is accepted. Table 6 gives the other words of the lecture.

Word	States visited	Accepted
01	Just1, Read0, Read01	yes
1010	Just1, Just1, Read0, Read01, Read01	yes
100011	Just1, Just1, Read0, Read0, Read0, Read01, Read01	yes
0	Just1, Read0	no
111000	Just1, Just1, Just1, Just1, Read0, Read0, Read0	no

Table 6: Runs of the automaton of Figure 8.

Automata in Rascal. In the code for this section an automaton is a set of initial states, a set of final states and a relation of transitions. The empty string `""` is used as the label of a λ-transition, which is needed in the next subsection. A DFA is the case with one initial state and one transition per state and symbol.

```
alias Trans = rel[str from, str sym, str to];

data Automaton = automaton(set[str] initials, set[str] finals, Trans delta);
```

The automaton of Figure 8 is written as

```
Automaton contains01 = automaton({"Just1"}, {"Read01"}, {
  <"Just1", "1", "Just1">,  <"Just1", "0", "Read0">,
  <"Read0", "0", "Read0">,  <"Read0", "1", "Read01">,
  <"Read01", "0", "Read01">, <"Read01", "1", "Read01">
});
```

and `Automata.rsc` checks it against the words of Table 6 and against every word of length up to 8.

3.3 Nondeterministic finite automata

DFA definitions are restrictive: every state needs exactly one transition for every symbol, and there is one initial state. A nondeterministic automaton relaxes three things.

1. Transitions may be over words, including the empty word λ. A transition labeled *abb* reads three symbols; a transition labeled λ reads nothing.

2. There is choice: a state may have several transitions with the same label, or none.

3. There may be several initial states.

Definition 3.5. A **nondeterministic finite automaton** (NFA) is a 5-tuple $\mathcal{A} = \langle Q, \Sigma, I, F, \Delta \rangle$ where $I \subseteq Q$ is a set of initial states and $\Delta \subseteq Q \times (\Sigma \cup \{\lambda\}) \times Q$ is a transition relation. Equivalently, Δ is a function from $Q \times (\Sigma \cup \{\lambda\})$ to $\mathcal{P}(Q)$, the set of subsets of Q.

Definition 3.6. A word $\omega = \sigma_1 \cdots \sigma_n$ is **recognized** by an NFA if there is a path from some initial state $q_I \in I$ to some final state $f \in F$ that reads all the symbols of ω, in order, possibly taking λ-transitions in between. We write $q_I \overset{\omega}{\Longrightarrow} f$.

The word is accepted if one path works; other paths may get stuck or end in non-final states. A DFA is an NFA with $I = \{q_I\}$, no λ-transitions and exactly one transition per state and symbol.

Example: last symbol 0, or only a 1. The lecture asks for an automaton over $\{0, 1\}$ that recognizes the words whose last symbol is 0 and the word 1. With λ-transitions the two cases can be built separately and joined at the start (Figure 9). From A the automaton chooses, without reading anything, which case it is in. The branch through B reads exactly one 1. The branch through C reads any word and then guesses that the current 0 is the last symbol. If the guess is wrong, that path gets stuck in D, which has no outgoing transitions.

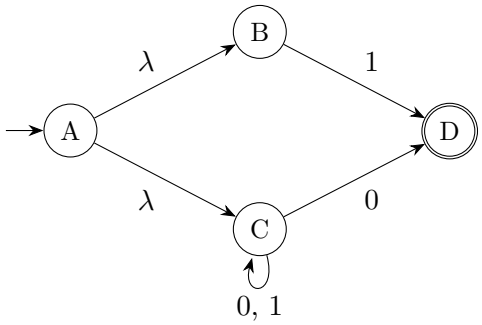


Figure 9: NFA for the words that end in 0, plus the word 1.

Example: `cat` and `hat`. Over $\{c, a, t, h\}$ the lecture defines the NFA of Figure 10. It has two branches joined by λ-transitions from A. The upper branch loops in B on every symbol, guesses where `hat` starts, reads it and stays in E. It accepts the words that contain `hat`. The lower branch is a DFA that remembers how much of `cat` it has just read: G after `c`, H after `ca`. A `t` in H means the word contains `cat`, and I is a trap state that is not final. F, G and H are final, so the lower branch accepts the words that do not contain `cat`.

Together, the automaton accepts the words that do not contain `cat` or that contain `hat`. The slide states the exercise as "does not contain the string cat or the string hat", which can also be read as "contains neither". Under that reading the automaton is wrong, since it accepts `hat` and `cathat`. Exercise 3.13 builds the automaton for the second reading. In Rascal:

```
Automaton catHat = automaton({"A"}, {"E", "F", "G", "H"}, {
  <"A", "", "B">,
  <"B", "c", "B">, <"B", "a", "B">, <"B", "t", "B">, <"B", "h", "B">,
  <"B", "h", "C">, <"C", "a", "D">, <"D", "t", "E">,
```

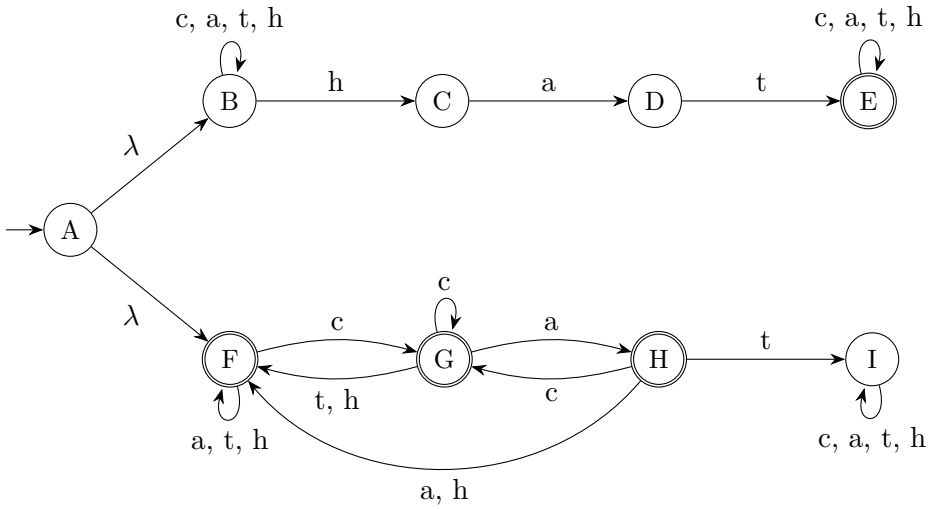


Figure 10: The NFA of the lecture over $\{c, a, t, h\}$.

```
  <"E", "c", "E">, <"E", "a", "E">, <"E", "t", "E">, <"E", "h", "E">,
  <"A", "", "F">,
  <"F", "a", "F">, <"F", "t", "F">, <"F", "h", "F">, <"F", "c", "G">,
  <"G", "t", "F">, <"G", "h", "F">, <"G", "c", "G">, <"G", "a", "H">,
  <"H", "a", "F">, <"H", "h", "F">, <"H", "c", "G">, <"H", "t", "I">,
  <"I", "c", "I">, <"I", "a", "I">, <"I", "t", "I">, <"I", "h", "I">
});
```

Running an NFA means keeping the set of states where some path could be. Before reading and after every symbol, the set is closed under λ-transitions:

```
set[str] closure(Trans delta, set[str] qs) {
  solve (qs) {
    qs += { t | <q, "", t> <- delta, q in qs };
  }
  return qs;
}

set[str] step(Trans delta, set[str] qs, str s)
  = closure(delta, { t | <q, s, t> <- delta, q in qs });

bool accepts(Automaton a, list[str] word) {
  set[str] current = closure(a.delta, a.initials);
  for (s <- word) {
    current = step(a.delta, current, s);
  }
  return current & a.finals != {};
}
```

3.4 From NFA to DFA

Nondeterminism makes automata easier to write, but it does not let them recognize more languages.

Theorem 3.7. *Every NFA has an equivalent DFA.*

The construction removes the three relaxations one at a time. All three use the same idea: a state of the new automaton stands for a set of states of the old one, and it behaves like all of them together. A set is initial or final according to the states it contains.

Transitions over words. A transition $\langle q, abb, q' \rangle$ becomes a chain $q \xrightarrow{a} p_1 \xrightarrow{b} p_2 \xrightarrow{b} q'$ through new states p_1, p_2.

Eliminate λ-transitions. The **λ-closure** $cl_\lambda(q)$ is the set of states reachable from q using only λ-transitions, including q itself. If the automaton can go from u to q reading a, and $t \in cl_\lambda(q)$, then it can also go from u to t reading a: add $\langle u, a, t \rangle$. A state becomes final if its closure contains a final state, and the initial states are replaced by their closures. After that, the λ-transitions can be deleted.

Eliminate choice. If a state has several transitions with the same symbol, replace the targets by one state that stands for the set of them. This is the **set construction**. Starting from the set of initial states, for each set S and symbol σ the next set is

$$\delta'(S, \sigma) = \bigcup_{q \in S} cl_\lambda(\{t \mid \langle q, \sigma, t \rangle \in \Delta\}),$$

and S is final if $S \cap F \neq \emptyset$. Only the sets reachable from the start are generated. The empty set appears when no state has a transition for σ; it is a trap state.

Remove additional initial states. The set I of initial states becomes a single initial state of the new automaton. In the set construction this happens by starting from $cl_\lambda(I)$ instead of from one state.

In Rascal the whole construction is a search over sets of states. Each set is named by listing its states, so `{A,S}` is a state of the DFA:

```
str name(set[str] qs) = "{" + intercalate(",", sort(toList(qs))) + "}";

Automaton determinize(Automaton a, set[str] sigma) {
  set[str] first = closure(a.delta, a.initials);
  list[set[str]] pending = [first];
  set[set[str]] seen = {first};
  Trans delta = {};
  while (pending != []) {
    set[str] current = head(pending);
    pending = tail(pending);
    for (s <- sigma) {
      set[str] next = step(a.delta, current, s);
      delta += <name(current), s, name(next)>;
      if (next notin seen) {
        seen += {next};
        pending += [next];
      }
    }
  }
  set[str] finals = { name(qs) | qs <- seen, qs & a.finals != {} };
```

```
  return automaton({name(first)}, finals, delta);
}
```

Example. Exercise 6 of the lecture asks to convert the NFA on the left of Figure 11 to a DFA. S reads a and may stay in S or go to A; it reads b and may go to B or C. Table 7 runs the set construction from $\{S\}$, and the DFA is on the right of the figure.

Set	on a	on b	Final
$\{S\}$	$\{A, S\}$	$\{B, C\}$	no
$\{A, S\}$	$\{A, C, S\}$	$\{B, C\}$	no
$\{B, C\}$	$\{C\}$	$\emptyset$	yes
$\{A, C, S\}$	$\{A, C, S\}$	$\{B, C\}$	yes
$\{C\}$	$\{C\}$	$\emptyset$	yes
$\emptyset$	$\emptyset$	$\emptyset$	no

Table 7: Set construction for the NFA of Figure 11.

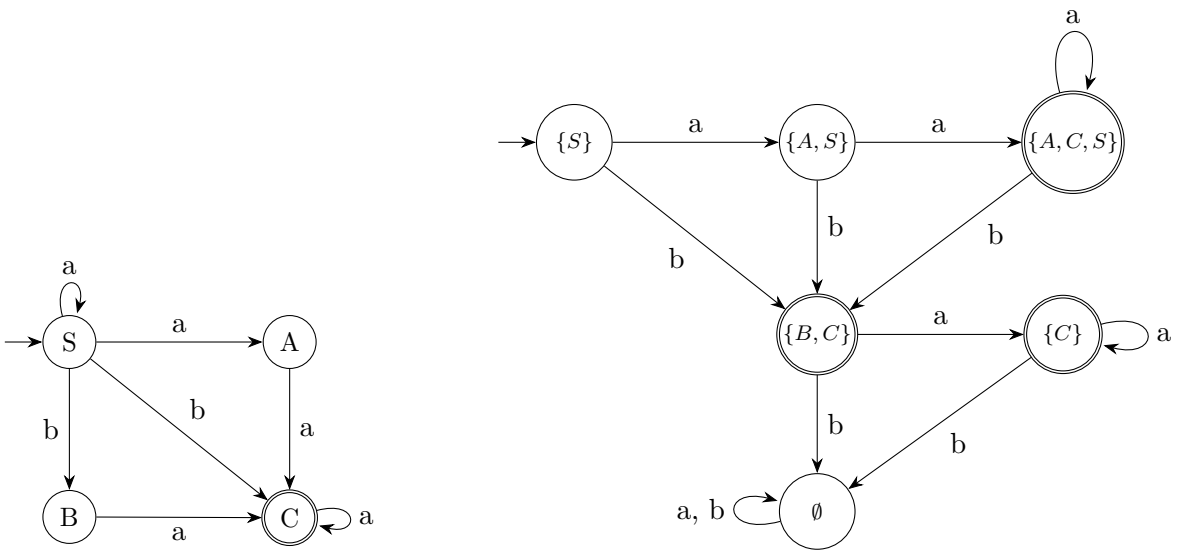


Figure 11: The NFA of exercise 6 (left) and the DFA produced by the set construction (right).

Six sets appear, out of the $2^4 = 16$ subsets of $\{S, A, B, C\}$. In the worst case an NFA with n states needs a DFA with 2^n states, but usually only a few sets are reachable. The tests in `Automata.rsc` check that the construction produces exactly these six states and that both automata accept the same words.

3.5 Combining automata

Automata for simple languages can be combined into automata for more complex ones. The lecture uses two automata over $\{a, b\}$ as running examples (Figure 12). L_b recognizes the words that contain a b. L_{2a} recognizes the words that end in aa: A loops on every symbol and guesses where the final aa starts. The λ back from C lets the automaton continue after an aa, but since only C is final, the word must end there.

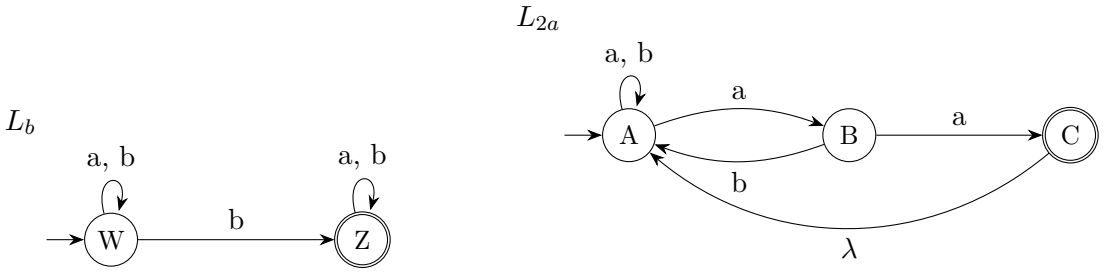


Figure 12: The two automata used for the combinations.

For $\mathcal{A}_1 = \langle Q_1, \Sigma, I_1, F_1, \Delta_1 \rangle$ and $\mathcal{A}_2 = \langle Q_2, \Sigma, I_2, F_2, \Delta_2 \rangle$ with disjoint sets of states:

Union. Add a new initial state q with a λ-transition to every initial state of both automata:

$$\mathcal{A}_1 \cup \mathcal{A}_2 = \langle Q_1 \cup Q_2 \cup \{q\}, \Sigma, \{q\}, F_1 \cup F_2, \Delta \rangle, \quad \Delta = \Delta_1 \cup \Delta_2 \cup \{\langle q, \lambda, i \rangle \mid i \in I_1 \cup I_2\}.$$

From q the automaton chooses which of the two to run.

Concatenation. Connect every final state of the first automaton to every initial state of the second:

$$\mathcal{A}_1\mathcal{A}_2 = \langle Q_1 \cup Q_2, \Sigma, I_1, F_2, \Delta \rangle, \quad \Delta = \Delta_1 \cup \Delta_2 \cup \{\langle f, \lambda, i \rangle \mid f \in F_1,\ i \in I_2\}.$$

For $L_b L_{2a}$ the automaton reads a prefix that contains a b, jumps from Z to A and then reads a suffix that ends in aa.

Kleene star. The star must accept the empty word and any sequence of words of $L(\mathcal{A}_1)$. The slide gives

$$\mathcal{A}_1^* = \langle Q_1, \Sigma, I_1, F_1 \cup I_1, \Delta \rangle, \quad \Delta = \Delta_1 \cup \{\langle f, \lambda, i \rangle \mid f \in F_1,\ i \in I_1\}$$

(the slide writes F_2 and Δ_2, but there is only one automaton). The λ-transitions from the final states back to the initial states allow repetition. Making the initial states final, to accept λ, does not work in general. In L_b the initial state W has a loop on a, so the word a would end in W and be accepted, but $a \notin L_b^*$: it is not empty and it contains no b. The safe construction adds a new initial state q that is final, with λ-transitions from q to the old initial states and from the old final states back to q:

$$\mathcal{A}_1^* = \langle Q_1 \cup \{q\}, \Sigma, \{q\}, F_1 \cup \{q\}, \Delta \rangle, \quad \Delta = \Delta_1 \cup \{\langle q, \lambda, i \rangle \mid i \in I_1\} \cup \{\langle f, \lambda, q \rangle \mid f \in F_1\}.$$

No old transition enters q, so only the empty word ends there without reading a word of $L(\mathcal{A}_1)$.

The three constructions in Rascal (`rename` prefixes the states of each automaton so that they are disjoint):

```
Automaton union(Automaton a1, Automaton a2) {
  a1 = rename(a1, "1"); a2 = rename(a2, "2");
  return automaton({"q"}, a1.finals + a2.finals,
    a1.delta + a2.delta + { <"q", "", i> | i <- a1.initials + a2.initials });
}
```

```
Automaton concat(Automaton a1, Automaton a2) {
  a1 = rename(a1, "1"); a2 = rename(a2, "2");
  return automaton(a1.initials, a2.finals,
    a1.delta + a2.delta + { <f, "", i> | f <- a1.finals, i <- a2.initials });
}

Automaton star(Automaton a) {
  a = rename(a, "1");
  return automaton({"q"}, a.finals + {"q"},
    a.delta + { <"q", "", i> | i <- a.initials } + { <f, "", "q"> | f <- a.finals });
}
```

Product construction. Intersection and difference cannot be built with λ-transitions, because the automaton has to read the word with both automata at the same time. For two DFAs $\mathcal{A}_1 = \langle Q_1, \Sigma, q_{I_1}, F_1, \delta_1 \rangle$ and $\mathcal{A}_2 = \langle Q_2, \Sigma, q_{I_2}, F_2, \delta_2 \rangle$, the product automaton has pairs of states:

$$\langle Q_1 \times Q_2, \Sigma, (q_{I_1}, q_{I_2}), F, \delta \rangle, \qquad \delta((p,q),\sigma) = (\delta_1(p,\sigma), \delta_2(q,\sigma)).$$

Only the final states change with the operation:

Operation	F
$L_1 \cup L_2$	$\{(p,q) \mid p \in F_1 \text{ or } q \in F_2\}$
$L_1 \cap L_2$	$\{(p,q) \mid p \in F_1 \text{ and } q \in F_2\}$
$L_1 \setminus L_2$	$\{(p,q) \mid p \in F_1 \text{ and } q \notin F_2\}$

Both automata must be complete DFAs: if one of them got stuck, the pair would have no next state. That is why the set construction keeps the empty set as a trap state.

Applying the set construction to L_b gives two states: $N = \{W\}$ (no b yet) and $Y = \{W, Z\}$ (a b was read). Applying it to L_{2a} gives three: $s_0 = \{A\}$, $s_1 = \{A, B\}$ (the word ends in one a) and $s_2 = \{A, B, C\}$ (it ends in aa). Figure 13 shows their product. For $L_b \cap L_{2a}$ the only final state is (Y, s_2); for $L_{2a} \setminus L_b$ it is (N, s_2); for the union it is every pair with Y or s_2.

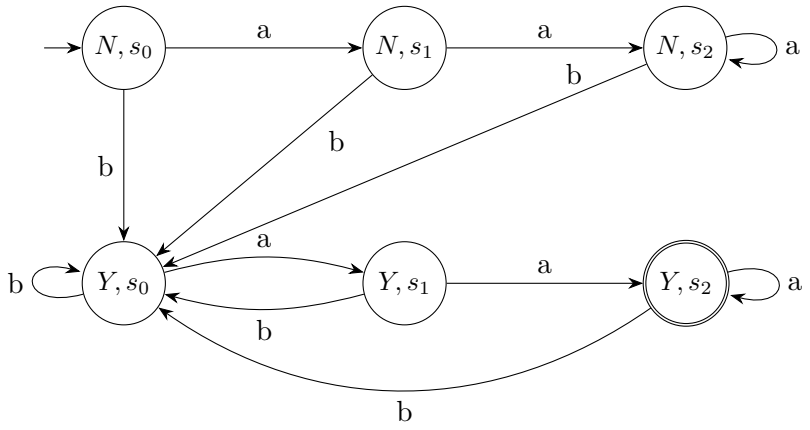


Figure 13: Product of the DFAs for L_b and L_{2a}. The final state shown is the one for $L_b \cap L_{2a}$.

In Rascal the product is built by searching the reachable pairs, and a function decides which pairs are final:

```
Automaton product(Automaton d1, Automaton d2, set[str] sigma, bool(bool, bool) keep)
    {
  map[tuple[str, str], str] t1 = ( <q, s> : t | <q, s, t> <- d1.delta );
  map[tuple[str, str], str] t2 = ( <q, s> : t | <q, s, t> <- d2.delta );
  str pair(str p, str q) = "(<p>,<q>)";
  tuple[str, str] first = <getOneFrom(d1.initials), getOneFrom(d2.initials)>;
  list[tuple[str, str]] pending = [first];
  set[tuple[str, str]] seen = {first};
  Trans delta = {};
  while (pending != []) {
    <p, q> = head(pending);
    pending = tail(pending);
    for (s <- sigma) {
      tuple[str, str] next = <t1[<p, s>], t2[<q, s>]>;
      delta += <pair(p, q), s, pair(next[0], next[1])>;
      if (next notin seen) { seen += {next}; pending += [next]; }
    }
  }
  set[str] finals = { pair(p, q) | <p, q> <- seen, keep(p in d1.finals, q in d2.
    finals) };
  return automaton({pair(first[0], first[1])}, finals, delta);
}

Automaton intersection(Automaton a1, Automaton a2, set[str] sigma)
  = product(determinize(a1, sigma), determinize(a2, sigma), sigma,
            bool(bool x, bool y) { return x && y; });

Automaton difference(Automaton a1, Automaton a2, set[str] sigma)
  = product(determinize(a1, sigma), determinize(a2, sigma), sigma,
            bool(bool x, bool y) { return x && !y; });
```

The tests compare each combination of L_b and L_{2a} with its definition on every word of length up to 8. They also confirm that the star from the slide accepts a and the corrected one does not.

3.6 Exercises

Exercise 3.8. Define an automaton that accepts the words over $\{0, 1\}$ with an even number of 0s and an even number of 1s.

Solution. The automaton has to remember two parities, so it needs four states: EE, OE, EO and OO, where the first letter is the parity of the 0s and the second the parity of the 1s (Figure 14). A 0 flips the first letter and a 1 flips the second. EE is initial and final.

Exercise 3.9. Define an automaton that accepts the words of the language of lists of Section 2: `[]`, `1|[]`, `1|2|3|[]`, but not `[1,2,3]` or `1|[]|2|[]`.

Solution. The lecture's automaton is in Figure 15. From q_0 the word either starts the empty list with `[` or starts a number. In q_3 the automaton reads the rest of the number; `|` moves it to q_4, where another number or the final `[]` may follow. Only q_2 is final, and q_2 has no outgoing transitions, so

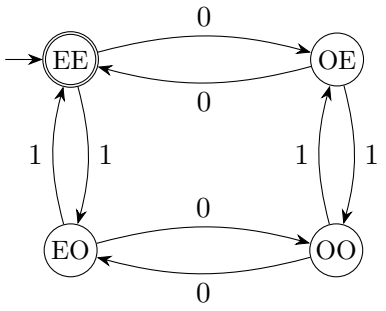


Figure 14: Even number of 0s and even number of 1s.

nothing can follow `[]`. The second version on the slide adds a state *err* and sends every missing transition there, which turns the automaton into a complete DFA.

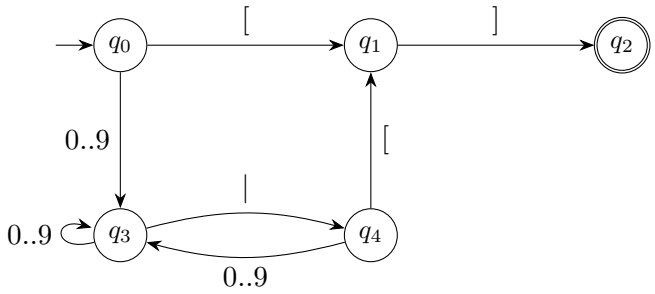


Figure 15: Automaton for the language of lists.

The Rascal version builds the transitions on digits with a comprehension:

```
set[str] digits = { "<i>" | i <- [0 .. 10] };

Automaton lists = automaton({"q0"}, {"q2"}, {
  <"q0", "[", "q1">, <"q1", "]", "q2">,
  *{ <"q0", d, "q3"> | d <- digits },
  *{ <"q3", d, "q3"> | d <- digits },
  <"q3", "|", "q4">,
  *{ <"q4", d, "q3"> | d <- digits },
  <"q4", "[", "q1">
});
```

Exercise 3.10. Write an NFA over $\{a, b\}$ that recognizes the words containing the pattern bb or bab exactly once (occurrences counted from left to right, without overlapping).

Solution. The lecture's automaton has initial state E, which loops on a until a b arrives. From A, a second b completes bb (through B and a λ to C) and a then b completes bab (through D). From C on, the automaton accepts while no second pattern appears: C loops on a, a single b goes to F, bb ends in the trap state H, and ba goes to G, where b would complete a second bab. The finals are C, F and G.

The automaton on the slide is missing one transition. After b and a it is in D, and D only has a transition on b. On the word $baabb$ the run is E, A, D and then stuck, so the word is rejected even

though it contains *bb* exactly once. The fix is $\langle D, a, E\rangle$: after *baa* no pattern has started, so the automaton goes back to the beginning (Figure 16).

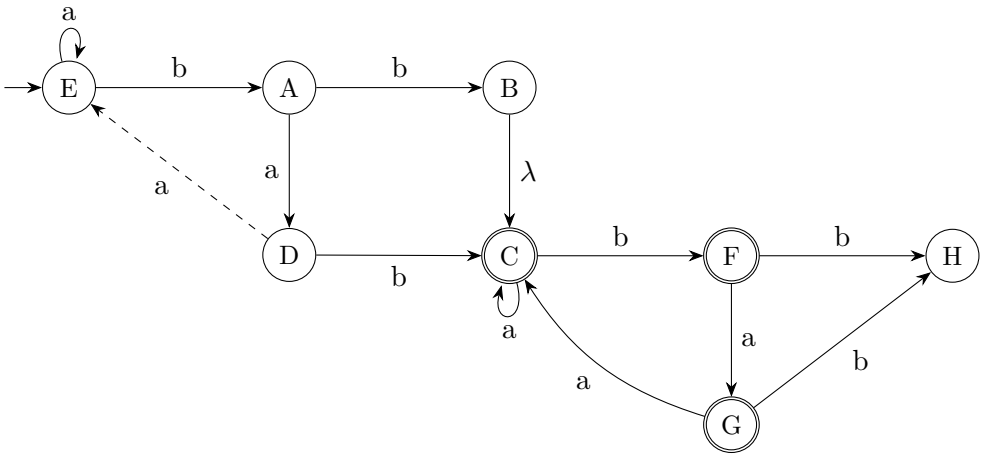


Figure 16: Exactly one *bb* or *bab*. The dashed transition is missing on the slide.

Exercise 3.11. Write an NFA that recognizes $(ab \mid aab \mid aba)^*$.

Solution. With transitions over words, the first step is a state E that is initial and final, with three loops labeled *ab*, *aab* and *aba*. Splitting the words into single symbols and sharing the common prefix *a* gives Figure 17: from E an *a* leads to X; from X, *b* returns to E (*ab*), *a* then *b* returns to E (*aab*), and *b* then *a* returns to E (*aba*). The automaton is nondeterministic only in X, which has two transitions on *b*.

The automaton on the slide puts a loop on *a* and *b* in E, makes C the only final state and adds λ-transitions from A and D to C. With that loop the automaton accepts every word that ends in *b* or *ba*, for example *b* and *bbb*, which are not in the language, and it rejects the empty word, which is.

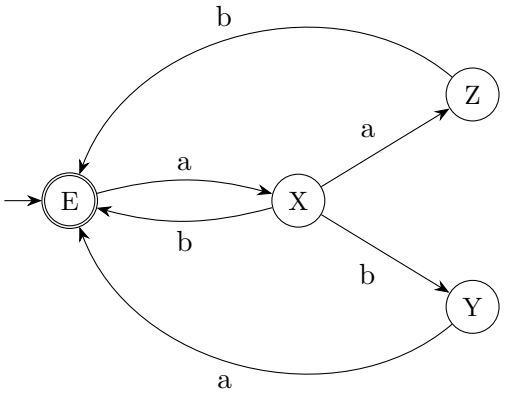


Figure 17: NFA for $(ab \mid aab \mid aba)^*$.

```
Automaton abStar = automaton({"E"}, {"E"}, {
  <"E", "a", "X">, <"X", "b", "E">,
  <"X", "a", "Z">, <"Z", "b", "E">,
```

```
  <"X", "b", "Y">, <"Y", "a", "E">
});
```

Exercise 3.12. Compute the λ-closure of each state of the automaton with transitions $A \xrightarrow{\lambda} B$, $A \xrightarrow{\lambda} C$, $B \xrightarrow{a} B$, $C \xrightarrow{a} D$, $D \xrightarrow{b} E$, $E \xrightarrow{\lambda} C$, with initial state A and final states B and E.

Solution. $cl_\lambda(A) = \{A, B, C\}$, $cl_\lambda(B) = \{B\}$, $cl_\lambda(C) = \{C\}$, $cl_\lambda(D) = \{D\}$ and $cl_\lambda(E) = \{E, C\}$. Eliminating the λ-transitions, A becomes final (its closure contains B), and every transition that enters E also enters C: $D \xrightarrow{b} C$ is added.

Exercise 3.13. Build a DFA over $\{c, a, t, h\}$ for the words that contain neither `cat` nor `hat`.

Solution. This is the intersection of two languages, so the product construction applies, but a direct construction is shorter. Both words end in `at`, so the automaton only needs to remember whether the last symbol was `c` or `h` (state 1) or whether the last two symbols were `ca` or `ha` (state 2). From state 0, `c` and `h` go to 1 and `a`, `t` stay in 0. From 1, `a` goes to 2, `c` and `h` stay in 1 and `t` goes back to 0. From 2, `t` goes to a trap state, `c` and `h` go to 1 and `a` goes back to 0. States 0, 1 and 2 are final.

4 Regular languages

Regular languages can be described in three ways that turn out to be equivalent: regular grammars generate them, regular expressions describe them and finite automata recognize them. This section follows Hopcroft, Motwani and Ullman [1, Ch. 3–4].

4.1 Regular grammars

Definition 4.1. A **regular grammar** (type 3) is a context-free grammar whose rules have one of two restricted forms. In a **right regular grammar** every rule is $A ::= a$ or $A ::= aB$; in a **left regular grammar** every rule is $A ::= a$ or $A ::= Ba$, with $A, B \in N$ and $a \in \Sigma$.

At most one nonterminal appears on the right side of a rule, always at the same end. That restriction is what makes the languages regular.

A right grammar. The lecture uses $\mathcal{G}_{3r} = \langle \{A, B\}, \{a, b\}, A, P_{3r} \rangle$ with the rules

$$A ::= a \qquad A ::= aB \qquad A ::= bA \qquad A ::= b \qquad B ::= bB \qquad B ::= a.$$

Grouping the rules of each nonterminal with `|` gives a shorter version,

```
A ::= a | aB | bA | b
B ::= bB | a
```

and with EBNF the recursion can be replaced by repetition: `bA` means any number of b's before the rest, and `bB` any number of b's before the final a.

```
A ::= b* (a | b | a B)
B ::= b* a
```

This last form is less common, but it shows the language directly: some b's, then a, or b, or an a followed by some b's and an a. As a regular expression, $b^*(a \mid b \mid ab^*a)$.

The word *bbbabba* is generated by a derivation in which every step adds one terminal at the left of the single nonterminal:

$$A \Rightarrow bA \Rightarrow bbA \Rightarrow bbbA \Rightarrow bbbaB \Rightarrow bbbabB \Rightarrow bbbabbB \Rightarrow bbbabba.$$

The lecture notes that this grammar defines a common data structure: a word of a right grammar is built one symbol at a time from the left, like a list.

A left grammar. The second example is $\mathcal{G}_{3l} = \langle \{S, A\}, \{a, b\}, S, P_{3l} \rangle$ with

$$S ::= Abbb \qquad S ::= Sb \qquad A ::= Aa \qquad A ::= aa.$$

The rules build the word from the right: S adds b's at the end and A adds a's at the end of the a part. The shortest word is $S \Rightarrow Abbb \Rightarrow aabbb$, and the language is aaa^*bbb^*: at least two a's followed by at least three b's.

Parse trees. A parse tree represents the structure of a word according to the production rules; the children of a node are the symbols of the right side of the rule applied. Figure 18 shows the trees of the two derivations above. A right grammar produces a tree that grows down to the right, a left grammar one that grows down to the left.

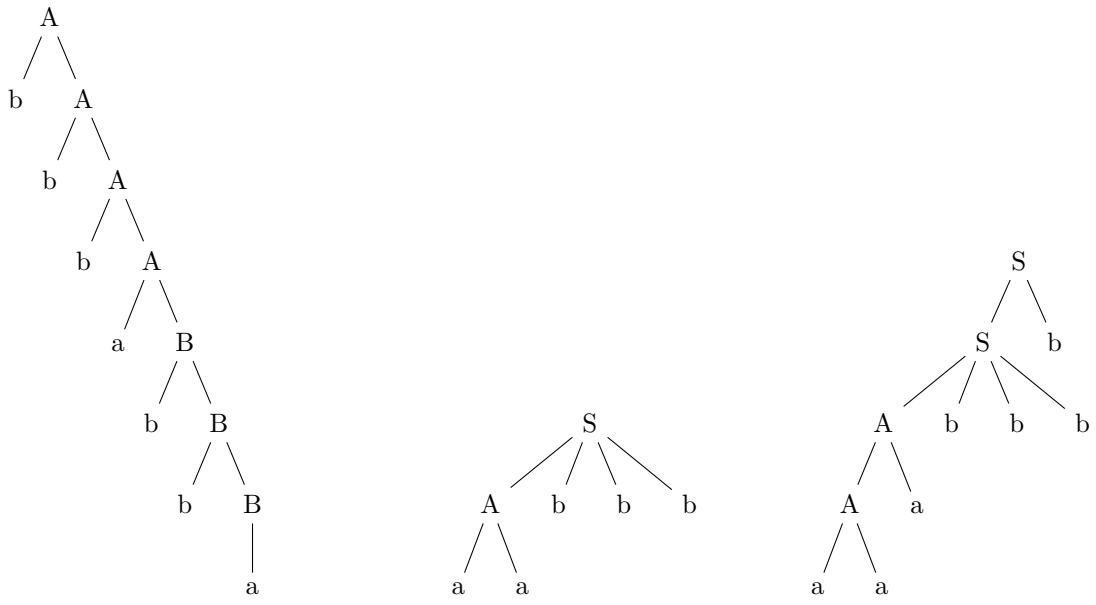


Figure 18: Parse trees for *bbbabba* in $\mathcal{G}_{3r}$ (left), and for *aabbb* and *aaabbbb* in $\mathcal{G}_{3l}$ (right).

Both grammars can be written in Rascal directly. They are `lexical` because no spaces may appear between the symbols:

```
lexical A = "a" | "a" B | "b" A | "b";
lexical B = "b" B | "a";
```

```
lexical S = L "bbb" | S "b";
lexical L = L "a" | "aa";
```

The tests compare each grammar with its regular expression on every word over $\{a, b\}$ of length up to 8.

From grammars to languages. The language of a grammar is the set of words of terminals that can be derived from the initial symbol,

$$L(\mathcal{G}) = \{\omega \in \Sigma^* \mid S_0 \Rightarrow^* \omega\},$$

where $\Rightarrow^*$ is a sequence of applications of the rules in P.

4.2 Regular expressions

Definition 4.2. The **regular expressions** over Σ are defined inductively:

- λ is a regular expression (the empty word);
- every $\sigma \in \Sigma$ is a regular expression;
- if α and β are regular expressions, so are $\alpha \mid \beta$ (union, or option), $\alpha\beta$ (concatenation) and α^* (Kleene star, zero or more repetitions).

Two abbreviations are common: $\alpha^+ = \alpha\alpha^*$ (one or more) and $\alpha? = \alpha \mid \lambda$ (optional, also written $[\alpha]$).

Regular expressions are a declarative way to describe the words of a regular language. They are used to recognize patterns in sequences of characters and to generate sequences that follow a pattern. In most libraries the dot is the pattern "any character", which is why the lecture's example `"one.two.".replaceAll(".", "*")` gives a different result depending on whether `"."` is treated as a literal string or as a regular expression.

Theorem 4.3. *Every regular language can be described by a regular expression, and the languages described by regular expressions are exactly those generated by regular grammars and recognized by finite automata.*

From regular expressions to grammars. The proof from expressions to grammars is a translation, case by case, shown in Table 8. Each row gives a right grammar and a left grammar with initial symbol S; for a composite expression, α and β stand for the grammars of the subexpressions.

For example, $(0 \mid 1)^*00(0 \mid 1)^*$ is a concatenation of three parts. With the rows for $(\alpha \mid \beta)^*$ and $\alpha\beta$:

```
S ::= 0S | 1S | 0A
A ::= 0B
B ::= 0B | 1B | λ
```

Expression	Right grammar	Left grammar
a	$S ::= a$	$S ::= a$
$\alpha \mid \beta$	$S ::= \alpha \mid \beta$	$S ::= \alpha \mid \beta$
$\alpha\beta$	$S ::= \alpha A$, $A ::= \beta$	$S ::= A\beta$, $A ::= \alpha$
α^*	$S ::= \alpha S \mid \lambda$	$S ::= S\alpha \mid \lambda$
α^+	$S ::= \alpha S \mid \alpha$	$S ::= S\alpha \mid \alpha$
$(\alpha \mid \beta)^*$	$S ::= \alpha S \mid \beta S \mid \lambda$	$S ::= S\alpha \mid S\beta \mid \lambda$
$(\alpha \mid \beta)^+$	$S ::= \alpha S \mid \beta S \mid \alpha \mid \beta$	$S ::= S\alpha \mid S\beta \mid \alpha \mid \beta$
$(\alpha\beta)^*$	$S ::= \alpha A \mid \lambda$, $A ::= \beta S$	$S ::= A\beta \mid \lambda$, $A ::= S\alpha$
$(\alpha\beta)^+$	$S ::= \alpha A$, $A ::= \beta S \mid \beta$	$S ::= A\beta$, $A ::= S\alpha \mid \alpha$

Table 8: Translation of regular expressions into right and left grammars.

S reads any prefix and then the first 0 of the pair, A reads the second 0, and B reads any suffix. The rule $B ::= \lambda$ is not of the form $A ::= a$. It can be removed by replacing it with $B ::= 0 \mid 1$ and adding $A ::= 0$, so that every word that used $B ::= \lambda$ ends with a terminal instead.

The other direction, from grammars to expressions, is much harder to do by hand. In practice one goes through the language or through the automaton.

4.3 Recognizing regular languages

Each regular grammar induces an NFA that recognizes the same language, and by Section 3 a DFA as well. For a right grammar the states are the nonterminals plus one final state f: a rule $A ::= aB$ becomes a transition $A \xrightarrow{a} B$ and a rule $A ::= a$ becomes $A \xrightarrow{a} f$. The lecture's example is $(0 \mid 1)^*1(0 \mid 1)$, the words whose second-to-last symbol is 1 (Figure 19).

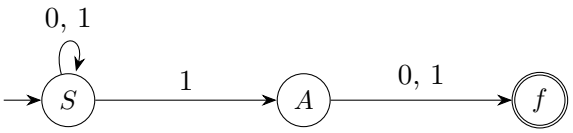


Figure 19: NFA for $(0 \mid 1)^*1(0 \mid 1)$, from the grammar $S ::= 0S \mid 1S \mid 1A$, $A ::= 0 \mid 1$.

Closure properties. The inductive definition gives the closure properties directly. The empty language and the language of one terminal are regular. If α and β describe regular languages, so do $\alpha \mid \beta$, $\alpha\beta$ and α^*. With the automata constructions of Section 3 the list grows: regular languages are closed under union, concatenation, Kleene star, intersection and difference, and the set construction (determinization) and minimization keep the language the same. The product construction is the reason intersection and difference belong to the list even though regular expressions have no operator for them.

4.4 A regular DSL: dates

The lecture builds a small domain-specific language to recognize dates, for example to validate values of a `Date` type. Three formats are allowed: `MM/DD/YYYY`, `DD-MM-YY` and `YYYY.MM.DD`. Months, days and years are numbers. The grammar is

```
Date   ::= Slash | Dash | Dot
Slash  ::= (Month | Month1) "/" (Day | Day1) "/" Year1 Year Year Year
Dash   ::= (Day | Day1) "-" (Month | Month1) "-" Year Year
Dot    ::= Year1 Year Year Year "." (Month | Month1) "." (Day | Day1)
Day    ::= (0..2)(0..9)
Day1   ::= 3(0 | 1)
Month  ::= 0(1..9)
Month1 ::= 1(0..2)
Year1  ::= (0..2)
Year   ::= (0..9)
```

`Month` covers 01 to 09 and `Month1` covers 10 to 12. `Day` covers 00 to 29 and `Day1` covers 30 and 31. A four-digit year starts with 0, 1 or 2. The parentheses around `Month | Month1` are needed: on the slide they are missing, and without them `|` would split the whole rule.

The word `03/08/1978` is generated as follows:

$$\begin{aligned} \mathit{Date} &\Rightarrow \mathit{Slash} \\ &\Rightarrow (\mathit{Month} \mid \mathit{Month1}) \ /\ (\mathit{Day} \mid \mathit{Day1}) \ /\ \mathit{Year1}\ \ \mathit{Year}\ \ \mathit{Year}\ \ \mathit{Year} \\ &\Rightarrow 03 \ /\ (\mathit{Day} \mid \mathit{Day1}) \ /\ \mathit{Year1}\ \ \mathit{Year}\ \ \mathit{Year}\ \ \mathit{Year} \\ &\Rightarrow 03 \ /\ 08 \ /\ \mathit{Year1}\ \ \mathit{Year}\ \ \mathit{Year}\ \ \mathit{Year} \\ &\Rightarrow 03 \ /\ 08 \ /\ 1\ \mathit{Year}\ \ \mathit{Year}\ \ \mathit{Year} \Rightarrow 03 \ /\ 08 \ /\ 19\ \mathit{Year}\ \ \mathit{Year} \\ &\Rightarrow 03 \ /\ 08 \ /\ 197\ \mathit{Year} \Rightarrow 03 \ /\ 08 \ /\ 1978 \end{aligned}$$

For recognition, `1979.10.01` is a `Dot` date, and `12-11-2025` is not a date of the language: `Dash` has a two-digit year, so after `12-11-20` the word should end.

Every rule is either a fixed set of symbols or a concatenation of such sets, so the language is regular. The lecture writes it as a single regular expression. The version on the slide has three problems: the month part is written $(0(1..9) \mid 1(0..2)0(1..9) \mid 1(0..2))$ instead of $(0(1..9) \mid 1(0..2))$, the day part appears twice in a row, and the day uses $(0..2)(1..9)$, which leaves out 10 and 20. The correct expression for the `Slash` format is

$$(0(1..9) \mid 1(0..2)) \ /\ ((0..2)(0..9) \mid 3(0 \mid 1)) \ /\ (0..2)(0..9)(0..9)(0..9)$$

and the other two formats are the same pieces in a different order.

The date DSL in Rascal. In Rascal each part of the date is `lexical`, so no spaces are allowed inside a date:

```
start syntax Date = Slash | Dash | Dot;

lexical Slash = MonthPart "/" DayPart "/" Year4;
lexical Dash  = DayPart "-" MonthPart "-" Year2;
lexical Dot   = Year4 "." MonthPart "." DayPart;

lexical DayPart   = Day | Day1;
lexical MonthPart = Month | Month1;

lexical Day    = [0-2] [0-9];
lexical Day1   = "3" [0-1];
lexical Month  = "0" [1-9];
```

```
lexical Month1 = "1" [0-2];
lexical Year4  = [0-2] [0-9] [0-9] [0-9];
lexical Year2  = [0-9] [0-9];
```

The slide's Rascal version uses `syntax` rules with a top-level `|`, as in

```
layout Spaces = [\ ]* !>> [\ ];
lexical Digit = [0-9];
lexical UptoTwo = [0-2];
syntax SlashSlide = "0" Digit | "1" UptoTwo "/" UptoTwo Digit
                  | "3" ("0" | "1") "/" UptoTwo Digit Digit Digit;
```

In Rascal, as in BNF, `|` separates the alternatives of the whole rule, so this rule says that a `SlashSlide` is either `"0" Digit`, or `"1" UptoTwo "/" UptoTwo Digit`, or the third alternative. The test `slideAcceptsMonthAlone` shows that `05` is accepted, and `slideRejectsFullDate` that `03/08/1978` is not. The slide also declares the layout as `WhitespaceAndeComment` and uses it as `WhitespaceAndComment`, so the module does not load.

The grammar does not know the calendar. It accepts day 00 and February 31, and it rejects month 13 and day 32 only because of the digits allowed in each position. Checking that a date exists needs arithmetic on the values, which is a job for the semantics of the DSL, not for its grammar.

4.5 Languages that are not regular

Lemma 4.4 (Pumping lemma)**.** *Let L be an infinite regular language. There is a constant k, which depends on L, such that every $\omega \in L$ with $|\omega| \geq k$ can be split as $\omega = \alpha\beta\gamma$ with $|\alpha\beta| \leq k$ and $\beta \neq \lambda$, and $\alpha\beta^n\gamma \in L$ for every $n \geq 0$.*

The idea comes from the automaton. If a DFA with k states reads a word of length at least k, some state repeats within the first k symbols. The part of the word read between the two visits, β, is a loop, and the automaton can go around it any number of times.

The lemma is a necessary condition. To show that a language is not regular, one shows that it fails the lemma, which works like a game against an opponent:

1. you are given the language L;
2. the opponent picks k;
3. you pick a word $\omega \in L$ with $|\omega| \geq k$, which may depend on k;
4. the opponent splits $\omega = \alpha\beta\gamma$ with $\beta \neq \lambda$ and $|\alpha\beta| \leq k$;
5. you pick n, which may depend on k, α, β and γ, such that $\alpha\beta^n\gamma \notin L$.

If you can always win, L is not regular.

Example: $L = \{a^m b^m \mid m \geq 0\}$ **is not regular.** Suppose L is regular and let k be the constant of the lemma. Take $\omega = a^k b^k$, which is in L and has length $2k \geq k$. Any split $\omega = \alpha\beta\gamma$ falls in one of the three cases of Figure 20. Only the first one is possible: $|\alpha\beta| \leq k$ and the first k symbols are a's, so $\alpha\beta$ contains only a's. (If β reached the b's, $|\alpha\beta|$ would exceed k.)

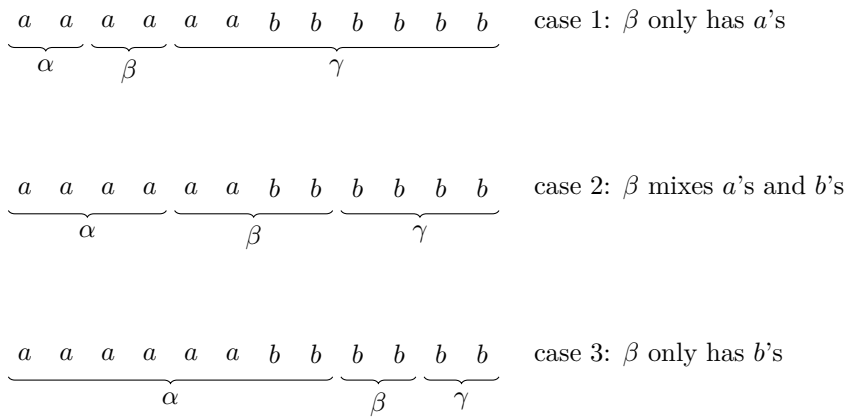


Figure 20: The three ways to split $a^k b^k$ (here $k = 6$). Cases 2 and 3 have $|\alpha\beta| > k$, so only case 1 is allowed.

So $\alpha = a^s$, $\beta = a^t$ and $\gamma = a^x b^k$, with $s + t + x = k$ and $t > 0$. The lemma says that $\alpha\beta^n\gamma = a^{s+nt+x}b^k \in L$ for every n, which requires $s + nt + x = k$. Take $n = 0$:

$$\begin{aligned} s + 0 \cdot t + x &= k && \langle n = 0 \rangle \\ s + x &= k && \langle \text{arithmetic} \rangle \\ s + x &= s + t + x && \langle \text{hypothesis } s + t + x = k \rangle \end{aligned}$$

which only holds if $t = 0$. But $t > 0$. This contradiction shows that the assumption was false, and L is not regular.

A consequence of the example is that regular languages cannot count: an automaton with finitely many states cannot remember how many a's it has read in order to compare with the b's. Languages that need this, such as balanced parentheses, need a larger class: the context-free languages of Section 5.

4.6 Regular expressions in practice

This subsection goes beyond the slides.

Thompson's construction. The proof that every regular expression has an automaton can be turned into an algorithm. Thompson's construction builds an NFA fragment with one initial and one final state for each case of the definition and glues the fragments with λ-transitions, the same way as the union, concatenation and star of Section 3. Figure 21 shows the result for a^*b. A person would draw two states; the construction is meant for a program, and minimizing the automaton is a separate step.

Regular expression libraries. The syntax of `grep`, Python's `re` or Java's `Pattern` has the same three operators plus abbreviations: character classes such as `[a-z]` are unions, `{2,}` is a bounded repetition and `\d` is `[0-9]`.

```
email_pattern = r"^[\w.+-]+@[\w-]+(\.[\w-]+)*\.[a-zA-Z]{2,}$"

log_pattern = r"^(\d{4}-\d{2}-\d{2}) (\d{2}:\d{2}:\d{2}) (\w+) (.+)$"
```

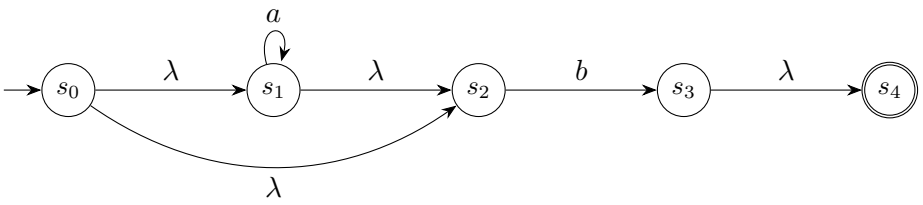


Figure 21: An NFA built by Thompson's construction for a^*b.

Some features of these libraries are not regular. A backreference such as `(\w+) \1` matches a word repeated twice, and to do that for words of any length an automaton would need to remember the whole first word. The test file shows it matching `is is`. Engines that support backreferences search by backtracking instead of simulating an automaton, and on some inputs they take exponential time, which a DFA never does.

Forward pointer. The tokens of a programming language (numbers, identifiers, keywords, operators) are regular languages, and a lexer is a finite automaton that recognizes all of them at once. The next block builds one.

4.7 Exercises

Exercise 4.5. Which of the following match $(A..G)^+(\flat \mid \sharp)?$: $A\flat$, $C\sharp$, $ABK\flat$, $A\flat B$, GFE?

Solution. $A\flat$, $C\sharp$ and GFE match. $ABK\flat$ does not, because K is not in $A..G$. $A\flat B$ does not either: the accidental can only appear once, at the end.

Exercise 4.6. Write regular expressions for: (a) "hello world" in English; (b) a consonant between every two pairs of vowels; (c) the words over $\{0,1\}$ that contain 00; (d) the words over $\{a,b\}$ where two a's never come together; (e) the words over $\{0,1\}$ that start with 0 and have odd length or start with 1 and have even length.

Solution. (a) The expression `hello world` itself: a fixed word is a concatenation of symbols. (b) With $V = (a \mid e \mid i \mid o \mid u)$ and $C = (b \mid c \mid d \mid \cdots \mid z)$ without the vowels, the lecture's answer is $(VVCVV)^*$. Two blocks next to each other give $VVCVVVVCVV$, with four vowels together; if the intended words are pairs of vowels separated by one consonant, the expression is $\lambda \mid VV(CVV)^*$. (c) $(0 \mid 1)^*00(0 \mid 1)^*$. (d) $a(b \mid ba)^* \mid (b \mid ba)^*$: every a after the first symbol is preceded by a b. (e) $0((0 \mid 1)(0 \mid 1))^* \mid 1(0 \mid 1)((0 \mid 1)(0 \mid 1))^*$. The answer on the slide ends with $1(0 \mid 1)^*$, which accepts the words that start with 1 of any length, for example 1 itself.

Exercise 4.7. Write a regular expression that checks that an e-mail address belongs to the domain `uniandes.edu.co`. Then write one for any e-mail address.

Solution. The slides write character sets with braces; in the usual syntax of regular expression libraries:

```
bool uniandes(str x) = /^[a-zA-Z0-9.\-]+@uniandes\.edu\.co$/ := x;
bool email(str x)    = /^[a-zA-Z0-9.\-]+@[a-zA-Z0-9\-]+(\.[a-zA-Z0-9\-]+)*\.(edu|com|
    co|net|org)$/ := x;
```

The braces on the slides mean repetition, as in EBNF, so the slide's version also accepts an address with nothing before the `@`; the `+` above requires at least one character. The dots of the domain are escaped because an unescaped dot means any character.

Exercise 4.8. Consider the grammar $s ::= (bc)^*t$, $b ::= m^+ \mid pbt$, $c ::= b \mid e^+$. What are the nonterminals, the terminals and the recursive rules? Is the grammar regular?

Solution. The nonterminals are the symbols that have rules: s, b and c. The terminals are t, m, p and e. The symbols `|`, `*` and `+` belong to the notation, not to the grammar. The only recursive rule is $b ::= pbt$. It has a nonterminal in the middle of its right side, so the grammar is not regular, and indeed b generates $p^n m^+ t^n$, which needs to count.

Exercise 4.9. For each language over $\{0,1\}$, write a regular expression and an automaton: (1) the number of 0s is divisible by 3; (2) even number of 0s; (3) the fifth symbol from the right is 1; (4) each 0 is followed by at least one 1; (5) 0^n1^m with $n \geq 3$ and m even; (6) at most two 0s; (7) start with 0 and have odd length or start with 1 and have even length; (8) any word except 0 and 1; (9) words that do not end with 01.

Solution. Table 9 gives the expressions and Figures 22–24 the automata. All nine expressions and automata are in `Regular.rsc`, and the tests compare them with a direct definition of each language on every word of length up to 9.

	Language	Regular expression
1	0s divisible by 3	$(1 \mid 01^*01^*0)^*$
2	even number of 0s	$(1 \mid 01^*0)^*$
3	fifth from the right is 1	$(0 \mid 1)^*1(0 \mid 1)(0 \mid 1)(0 \mid 1)(0 \mid 1)$
4	each 0 followed by a 1	$(0?1)^*$
5	0^n1^m, $n \geq 3$, m even	$000^*0(11)^*$
6	at most two 0s	$1^*0?1^*0?1^*$
7	odd with 0, even with 1	$(0 \mid 1(0 \mid 1))((0 \mid 1)(0 \mid 1))^*$
8	not 0, not 1	$\lambda \mid (0 \mid 1)(0 \mid 1)(0 \mid 1)^*$
9	not ending in 01	$\lambda \mid 0 \mid 1 \mid (0 \mid 1)^*(0 \mid 1(0 \mid 1))$

Table 9: Regular expressions for the nine languages.

A few notes. In (1) each group of three 0s may have 1s between them, and the outer star allows 1s anywhere else. In (5) the expression 000^*0 is the lecture's way of writing at least three 0s (00, then 0^*, then 0). The slide's expressions for (8) and (9) leave out λ, which belongs to both languages: the empty word is neither 0 nor 1, and it does not end with 01. The slide's automaton for (9) has the transitions $0 \xrightarrow{1} 0$, $0 \xrightarrow{0} 0$, $0 \xrightarrow{0} 1$, $0 \xrightarrow{1} 2$, $2 \xrightarrow{1} 3$ and final states $\{1,2,3\}$. It accepts 01: from 0 it reads 0 and stays, then reads 1 and goes to the final state 2. The automaton of Figure 24 remembers the last two symbols instead.

Exercise 4.10. Is `12-11-2025` a word of the date language? And `02/31/2000`? Change the grammar so that `Dash` also accepts four-digit years.

Solution. `12-11-2025` is not: `Dash` ends after two year digits. `02/31/2000` is, even though the date does not exist. Replacing the end of `Dash` with `(Year Year | Year1 Year Year Year)` accepts both two- and four-digit years.

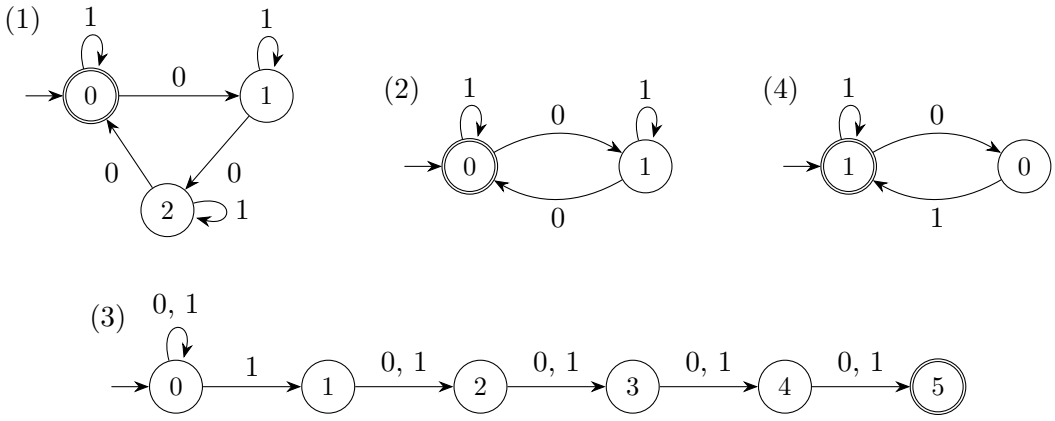


Figure 22: Automata for languages (1), (2), (4) and (3). The automaton for (3) is nondeterministic: in state 0 it guesses that the current 1 is the fifth symbol from the end. In (4) a 0 with no 1 after it gets stuck.

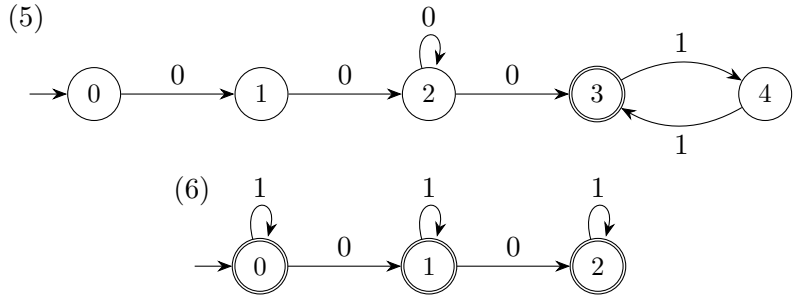


Figure 23: Automata for languages (5) and (6). In (5) the loop on state 2 makes the automaton nondeterministic; in (6) a third 0 gets stuck.

Exercise 4.11. Use the pumping lemma to show that $\{0^n 1 0^n \mid n \geq 0\}$ is not regular.

Solution. Given k, take $\omega = 0^k 1 0^k$. Since $|\alpha\beta| \leq k$, $\beta = 0^t$ with $t > 0$ lies in the first block of 0s. With $n = 0$ the word becomes $0^{k-t}10^k$, which has fewer 0s before the 1 than after it, so it is not in the language.

5 Context-free languages

Regular languages cannot count (Section 4), and the syntax of programming languages is full of things that must be counted: parentheses, `begin` and `end`, nested blocks. Context-free languages can. This section follows Hopcroft, Motwani and Ullman [1, Ch. 5–7].

5.1 Context-free grammars

Definition 5.1. A **context-free grammar** (type 2) is a grammar in which the left side of every rule is a single nonterminal: every rule has the form $A ::= \beta$ with $A \in N$ and $\beta \in (N \cup \Sigma)^*$.

The name says that A can be replaced by β regardless of what surrounds A in the sentential form. Context-free grammars are a subset of the context-sensitive grammars (type 1), and they

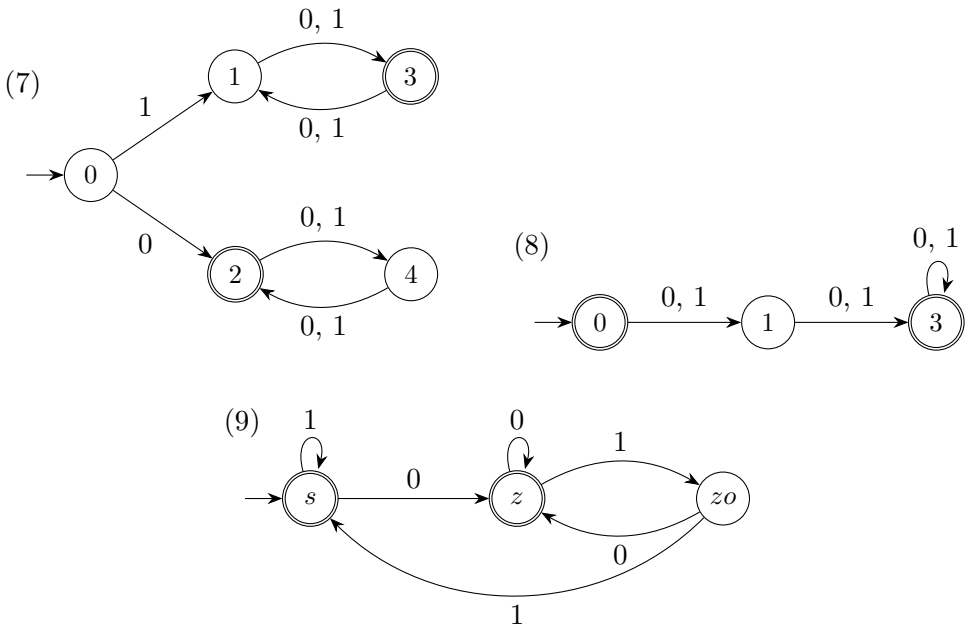


Figure 24: Automata for languages (7), (8) and (9). In (7) the states 1 and 3 count parity after a first 1, and 2 and 4 after a first 0. In (9) the state z means "the last symbol is 0" and zo means "the last two symbols are 01".

allow $\beta = \lambda$. Unlike regular grammars, the right side can mix blocks of terminals and nonterminals in any order.

Example: equal numbers of a's and b's. The lecture's grammar $\mathcal{G}_{2'} = \langle \{S, A, B\}, \{a, b\}, S, P_{2'} \rangle$ has the rules

$$S ::= aB \mid bA \mid \lambda \qquad A ::= aS \mid bAA \qquad B ::= bS \mid aBB.$$

A generates the words with one a more than b's, and B those with one b more. S starts with one letter and asks for a word that compensates it. The word $abaabb$ has the derivation

$$S \Rightarrow aB \Rightarrow abS \Rightarrow abaB \Rightarrow abaaBB \Rightarrow abaaBbS \Rightarrow abaaBb \Rightarrow abaabSb \Rightarrow abaabb,$$

where the sixth and the last step use $S ::= \lambda$. (The slides write the steps with `::=`; a derivation step is $\Rightarrow$.) The language of $\mathcal{G}_{2'}$ is the set of words with as many a's as b's, which the pumping lemma shows is not regular.

Example: a grammar with three parts. The second example is $\mathcal{G}_2$, with nonterminals $\{S, A, B, C\}$, terminals $\{a, b, c\}$, initial symbol S and the rules

$$S ::= ABC \qquad A ::= aAb \mid B \qquad B ::= bB \mid \lambda \qquad C ::= ccC \mid B.$$

A generates $a^n b^j b^n$, B generates b^* and C generates $(cc)^k b^l$. The derivation of $aabbccccbbb$ in the lecture is

$$\begin{aligned} S &\Rightarrow ABC \Rightarrow aAbBC \Rightarrow aaAbbBC \Rightarrow aaAbbBccC \Rightarrow aaAbbBccccC \Rightarrow aaAbbccccC \\ &\Rightarrow aaAbbccccB \Rightarrow aaBbbccccB \Rightarrow aabbccccB \Rightarrow aabbccccbB \Rightarrow aabbccccbbB \Rightarrow aabbccccbbbB \\ &\Rightarrow aabbccccbbb. \end{aligned}$$

In Rascal both grammars can be written as given; an empty alternative is the rule with λ:

```
lexical S2 = "a" B2 | "b" A2 | ;
lexical A2 = "a" S2 | "b" A2 A2;
lexical B2 = "b" S2 | "a" B2 B2;
```

```
lexical S3 = A3 B3 C3;
lexical A3 = "a" A3 "b" | B3;
lexical B3 = "b" B3 | ;
lexical C3 = "cc" C3 | B3;
```

`CFL.rsc` checks that $\mathcal{G}_{2'}$ generates exactly the words with equal numbers of a's and b's up to length 8, and that $\mathcal{G}_2$ generates exactly the words $a^n b^m (cc)^k b^l$ with $m \geq n$ up to length 7.

5.2 Parse trees

Definition 5.2. A **parse tree** for a context-free grammar $\mathcal{G} = \langle N, \Sigma, S_0, P \rangle$ satisfies three conditions:

1. interior nodes are nonterminals;
2. leaves are terminals or λ;
3. an interior node labeled A has children $X_1, X_2, \ldots, X_n$ only if $A ::= X_1 X_2 \cdots X_n$ is a rule.

The **yield** of the tree is the concatenation of its leaves from left to right; it is always a word.

In a regular grammar the tree is a path with leaves on one side (Figure 18). In a context-free grammar a rule can create several nonterminals, and the tree branches. Figure 25 shows the tree of the derivation of *abaabb*; the rule $B ::= aBB$ is where it branches.

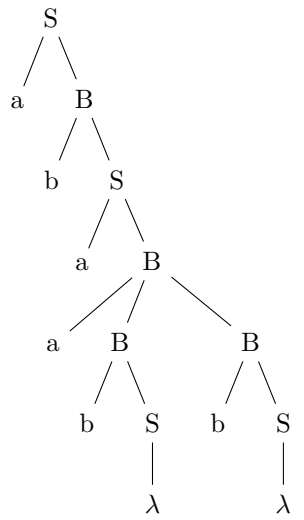


Figure 25: Parse tree of *abaabb* in $\mathcal{G}_{2'}$. Its yield is *abaabb*.

Example: Grace. The lecture shows a fragment of the Grace language, a method that counts down and doubles a variable:

```
method run {
    var y := 1
    var x := 4
    while {x > 0} do {
        x := x - 1
        y := y + y
    }
    print(x)
    print(y)
}
```

with this grammar for methods:

```
S     ::= method run { Stmts }
Stmts ::= Stmt \n Stmts | λ
Stmt  ::= var Ident := Exp
Stmt  ::= Exp
Stmt  ::= while { Exp } do { Stmts }
Exp   ::= Exp + Exp | Exp - Exp | Exp > Exp
Exp   ::= Ident | Num | print(Exp)
Ident ::= x | y
Num   ::= (0 | 1 | 4 | 3)+
```

The slide writes the second line of `Exp` as `Ident ::= Exp | Ident | Num | print(Exp)`; it has to be a rule of `Exp`, otherwise expressions could not contain variables, numbers or `print`. Two more rules are missing for the program to be generated: the assignment `x := x - 1` needs `Stmt ::= Ident := Exp`, and the line `run` that the slide writes after the method (a call) is not generated by `S`. Figure 26 shows the top of the parse tree.

5.3 Ambiguity

The lecture's expression language is

```
Exp ::= Exp + Exp
Exp ::= Exp * Exp
Exp ::= ( Exp )
Exp ::= (1 | 2 | 3 | 4)+
```

The word $2 + 3 * 1$ has two parse trees (Figure 27). The first treats it as a sum whose right operand is a product; the second as a product whose left operand is a sum. Each tree has its own leftmost derivation:

$$Exp \Rightarrow Exp + Exp \Rightarrow 2 + Exp \Rightarrow 2 + Exp * Exp \Rightarrow 2 + 3 * Exp \Rightarrow 2 + 3 * 1$$
$$Exp \Rightarrow Exp * Exp \Rightarrow Exp + Exp * Exp \Rightarrow 2 + Exp * Exp \Rightarrow 2 + 3 * Exp \Rightarrow 2 + 3 * 1$$

and its own rightmost derivation, for example for the first tree $Exp \Rightarrow Exp + Exp \Rightarrow Exp + Exp * Exp \Rightarrow Exp + Exp * 1 \Rightarrow Exp + 3 * 1 \Rightarrow 2 + 3 * 1$. Leftmost and rightmost derivations of the same tree are different sequences of steps, but they build the same tree. Ambiguity is about trees: a grammar is ambiguous when one word has two trees, which is the same as having two leftmost derivations.

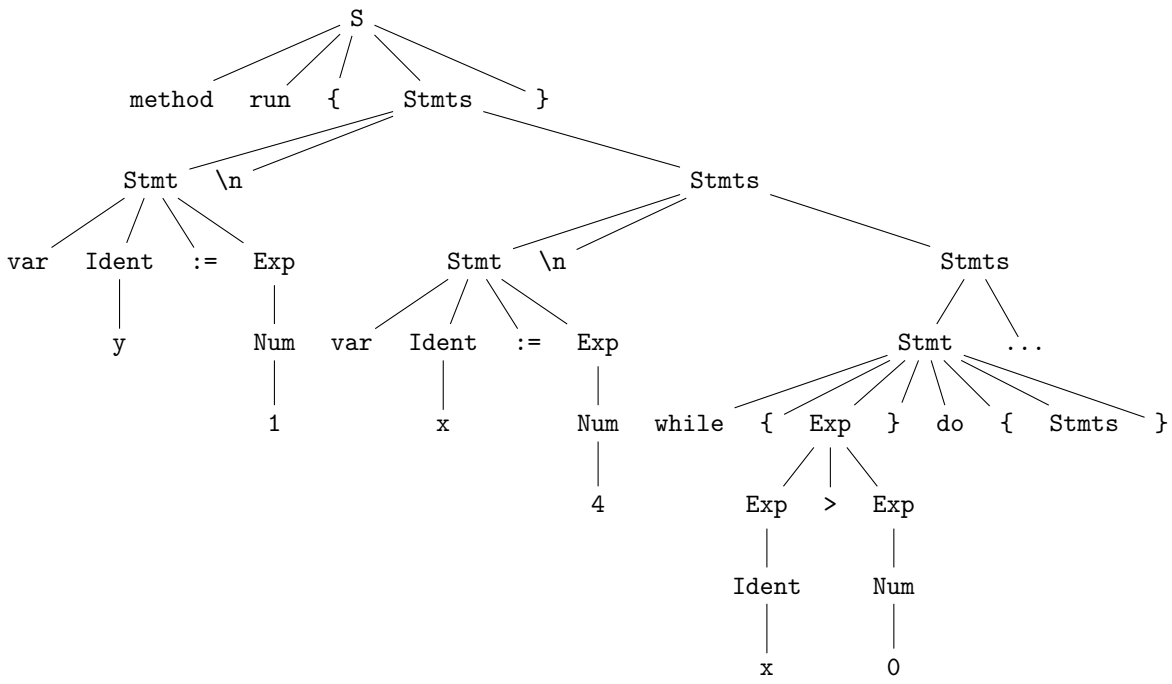


Figure 26: The top of the parse tree of the Grace method. The body of the `while` and the two `print` statements are omitted.

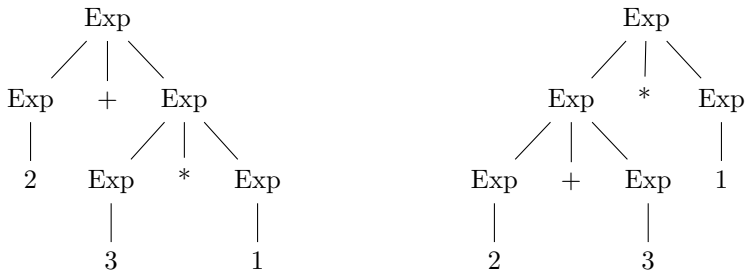


Figure 27: Two parse trees for $2 + 3 * 1$.

Ambiguity is a problem for compilers: the tree decides what the program means, and a compiler has no fast, deterministic way to choose. In Rascal the grammar can be written as it is, and the parser returns both trees:

```
syntax E
  = E "+" E
  | E "*" E
  | "(" E ")"
  | Num
  ;
```

5.4 Removing ambiguity

For the common constructs of programming languages, ambiguity is removed with two kinds of rules: operator precedence and associativity.

Precedence. Precedence is enforced through the grammar by adding one nonterminal per level:

```
Ident  ::= (0..9)+
Factor ::= Ident | ( Exp )
Term   ::= Factor | Term * Factor
Exp    ::= Term | Exp + Term
```

Factors cannot be broken by `*` or `+`, and terms cannot be broken by `+`. A `+` can only appear at the `Exp` level, so in $2 + 3 * 4$ the product must be a `Term` inside the sum (Figure 28).

Associativity. The rule `Exp ::= Exp + Term` has the recursion on the left, so in $1 + 2 + 3$ the left operand is $1 + 2$: the sum associates to the left. Writing `Exp ::= Term + Exp` would make it associate to the right.

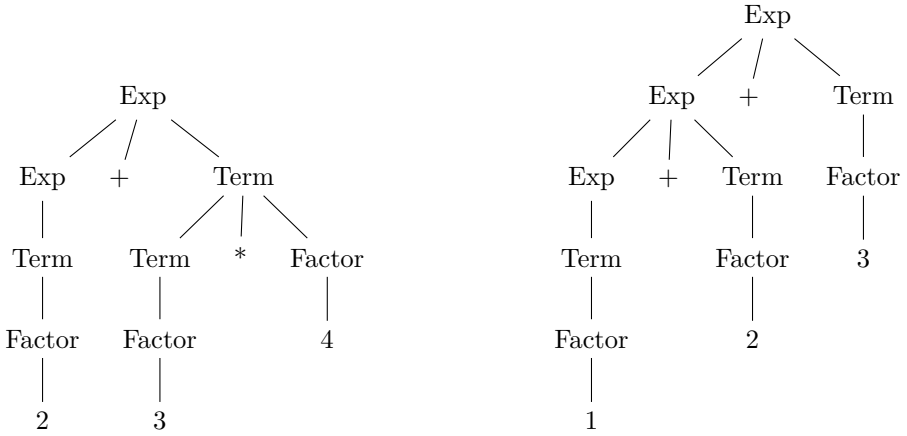


Figure 28: The only parse tree of $2 + 3 * 4$ (left) and of $1 + 2 + 3$ (right) in the grammar with precedence.

The grammar in Rascal, and an evaluator written with concrete syntax patterns, one equation per rule:

```
syntax Exp    = Exp "+" Term | Term;
syntax Term   = Term "*" Factor | Factor;
syntax Factor = "(" Exp ")" | Num;
```

```
int eval((Exp) `<Exp e> + <Term t>`)       = eval(e) + eval(t);
int eval((Exp) `<Term t>`)                 = eval(t);
int eval((Term) `<Term t> * <Factor f>`)   = eval(t) * eval(f);
int eval((Term) `<Factor f>`)              = eval(f);
int eval((Factor) `( <Exp e> )`)           = eval(e);
int eval((Factor) `<Num n>`)               = toInt("<n>");
```

The tests check that $2 + 3 * 4$ has a single tree and evaluates to 14, that $(2 + 3) * 4$ evaluates to 20, and that the left operand of the top `+` in $1 + 2 + 3$ is $1 + 2$.

The lecture adds that LR parsers give deterministic and fast algorithms for context-free grammars. They read the input from left to right and produce a rightmost derivation in reverse (the slide says leftmost; top-down LL parsers are the ones that produce leftmost derivations). Both kinds of parser are the subject of the next block.

Dangling else. The word `if x then if y then z else w` has two trees in the grammar

```
Stmt      ::= ... | Selection
Selection ::= ... | if ( Exp ) Stmt | if ( Exp ) Stmt else Stmt
```

because the `else` can belong to either `if` (Figure 29). The ambiguity is removed by requiring that the `then` branch of an `if` with `else` is always closed (matched):

```
Stmt       ::= OpenStmt | ClosedStmt
OpenStmt   ::= if ( Exp ) Stmt
             | if ( Exp ) ClosedStmt else OpenStmt
ClosedStmt ::= NonIfStmt
             | if ( Exp ) ClosedStmt else ClosedStmt
NonIfStmt  ::= ...
```

A closed statement has an `else` for every `if`. In the example, the inner `if y then z` has no `else` of its own, so it is open and cannot be the `then` branch of an `if` with `else`. The only tree left is the one where the `else` belongs to the inner `if`.

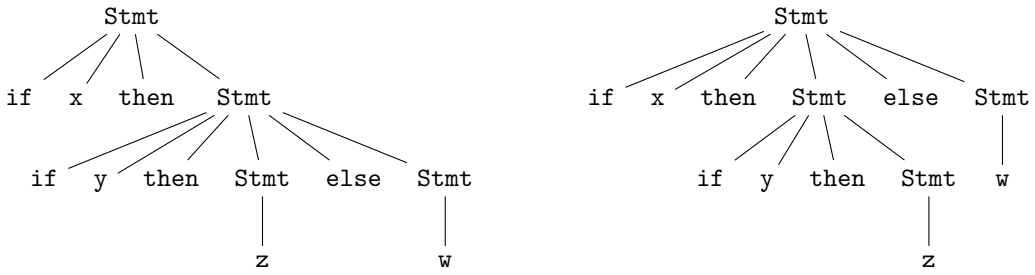


Figure 29: The two trees of `if x then if y then z else w`. The grammar with open and closed statements only allows the left one.

In Rascal, without parentheses around the condition:

```
syntax Stmt2     = OpenStmt | ClosedStmt;
syntax OpenStmt  = "if" Id "then" Stmt2
                 | "if" Id "then" ClosedStmt "else" OpenStmt;
syntax ClosedStmt = Id
                  | "if" Id "then" ClosedStmt "else" ClosedStmt;
```

5.5 Left recursion

Generating words of the precedence grammar from the left shows a problem:

$$Exp \Rightarrow Exp + Term \Rightarrow Exp + Term + Term \Rightarrow Exp + Term + Term + Term \Rightarrow \cdots$$

The leftmost symbol stays `Exp`, and a procedure that always expands it first never reads a terminal. Left recursion is difficult to manage for top-down parsers. It is eliminated by moving the recursion to the right with a new nonterminal: $A ::= A\gamma \mid \delta$ becomes $A ::= \delta A'$ and $A' ::= \gamma A' \mid \lambda$. For the expression grammar:

```
Exp    ::= Term Exp'
Exp'   ::= + Term Exp' | λ
```

```
Term   ::= Factor Term'
Term'  ::= * Factor Term' | λ
Factor ::= ( Exp ) | Num
Num    ::= (0..9)+
```

The language is the same; `CFL.rsc` checks it on every word of length up to 6 over $\{1, +, *, (,)\}$. The trees are different: in the new grammar the `+` hangs from `Exp'` on the right, so associativity has to be recovered when the tree is processed.

The lecture also gives the general algorithm, for recursion through several nonterminals ($A_1 \Rightarrow A_2\gamma \Rightarrow A_1\gamma'\gamma$). Sort the nonterminals as $A_1, \ldots, A_n$. For each i and each $j < i$, replace every rule $A_i ::= A_j\gamma$ by $A_i ::= \delta_1\gamma \mid \cdots \mid \delta_k\gamma$, where $A_j ::= \delta_1 \mid \cdots \mid \delta_k$ are the current rules of A_j; then eliminate the immediate left recursion of A_i. If the grammar has no cycles ($A \Rightarrow^+ A$) and no λ-rules, left recursion can always be eliminated.

5.6 Pushdown automata

A DFA cannot recognize $a^n b^n$: to check that there are exactly n characters of each kind, it has to remember how many a's it has read, and it has finitely many states. The lecture extends automata with memory: a stack. While reading *aaaabbbb* the automaton pushes one symbol per a and pops one per b. If the stack is empty exactly when the word ends, the counts are equal. Figure 30 shows the automaton and Table 10 the run.

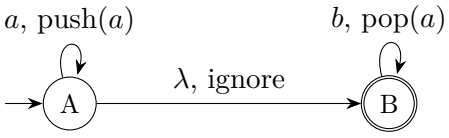


Figure 30: Pushdown automaton for $\{a^n b^n \mid n \geq 0\}$.

State	Stack (top first)	Input left	Transition
A	empty	*aaabbb*	a, push(a)
A	a	*aabbb*	a, push(a)
A	*aa*	*abbb*	a, push(a)
A	*aaa*	*bbb*	λ, ignore
B	*aaa*	*bbb*	b, pop(a)
B	*aa*	*bb*	b, pop(a)
B	a	b	b, pop(a)
B	empty	empty	accept

Table 10: Run of the automaton of Figure 30 on *aaabbb*.

Definition 5.3. A **pushdown automaton** (PDA) is a 6-tuple $\mathcal{A} = \langle Q, \Sigma, \Gamma, q_I, F, \Delta \rangle$ where Q is a set of states, Σ is the input alphabet, Γ is the stack alphabet, q_I is the initial state, $F \subseteq Q$ is the set of final states and Δ is a transition relation. A transition $((q, \sigma, \beta), (p, \varphi))$ reads: in state q, with β on top of the stack, reading $\sigma \in \Sigma \cup \{\lambda\}$ from the input, move to state p and leave φ on top of the stack instead of β.

The lecture names the stack operations that a transition can perform:

Operation	Effect
push(e)	put e on top of the stack
pop(e)	take e off the top of the stack (only if e is on top)
ignore	leave the stack as it is
changeTop(e, e')	replace the top e by e'
pushOn(e, e')	put e on top if the current top is e'

Definition 5.4. A **configuration** of a PDA is a triple (q, σ, ω): the current state, the contents of the stack and the input still to read. A configuration (q', σ', ω') is reached from (q, σ, ω), written $(q, \sigma, \omega) \vdash (q', \sigma', \omega')$, if a transition from q to q' reads the first symbol of ω (or nothing) and its stack operation turns σ into σ'. A word ω is **recognized** if $(q_I, \emptyset, \omega) \vdash^* (f, \emptyset, \lambda)$ for some $f \in F$: starting with an empty stack, the automaton reads the whole word and ends in a final state with an empty stack.

So three conditions must hold at the end: the word has been read completely, a final state has been reached, and the stack is empty. In Rascal a transition records what it reads, what it expects on top of the stack and what it puts there instead; each operation of the table is a special case:

```
alias PTrans = rel[str from, str read, str top, list[str] push, str to];

data PDA = pda(str initial, set[str] finals, PTrans delta);

alias Config = tuple[str state, list[str] stack, list[str] input];
```

A PDA is nondeterministic, so recognizing a word is a search over configurations:

```
bool recognizes(PDA m, list[str] word, int maxStack = size(word) + 3) {
  Config first = <m.initial, [], word>;
  list[Config] pending = [first];
  set[Config] seen = {first};
  while (pending != []) {
    Config c = head(pending);
    pending = tail(pending);
    if (c.state in m.finals && c.stack == [] && c.input == []) return true;
    for (<q, r, t, p, q2> <- m.delta, q == c.state) {
      if (r != "" && (c.input == [] || head(c.input) != r)) continue;
      if (t != "" && (c.stack == [] || head(c.stack) != t)) continue;
      list[str] rest = t == "" ? c.stack : tail(c.stack);
      Config next = <q2, p + rest, r == "" ? c.input : tail(c.input)>;
      if (size(next.stack) <= maxStack && next notin seen) {
        seen += {next};
        pending += [next];
      }
    }
  }
  return false;
}
```

The automaton of Figure 30:

```
PDA anbn = pda("A", {"B"}, {
  <"A", "a", "",  ["a"], "A">,
  <"A", "",  "",  [],    "B">,
```

```
  <"B", "b", "a", [],    "B">
});
```

From a grammar to a PDA. Context-free languages are exactly the languages recognized by pushdown automata, and every context-free grammar induces a PDA for its language. The automaton builds the word and recognizes it at the same time. The stack holds the part of the sentential form that still has to be matched against the input:

1. push a bottom marker \$ and the initial symbol;
2. if a nonterminal A is on top, replace it by the right side of one of its rules, λ, changeTop(A, β), with the first symbol of β on top;
3. if a terminal a is on top and a is the next input symbol, read it and pop it;
4. if \$ is on top, pop it and go to the final state.

All the work happens in one state q. Table 11 runs the automaton for $S ::= aSb \mid ab$ on $aabb$. The choice in the second row is nondeterministic: replacing S by ab there would fail at the next a.

Stack (top first)	Input left	Step
$S\,\$$	$aabb$	changeTop(S, aSb)
$aSb\,\$$	$aabb$	read a, pop(a)
$Sb\,\$$	abb	changeTop(S, ab)
$abb\,\$$	abb	read a, pop(a)
$bb\,\$$	bb	read b, pop(b)
$b\,\$$	b	read b, pop(b)
\$	empty	pop(\$), accept

Table 11: The PDA of $S ::= aSb \mid ab$ on $aabb$.

In Rascal the construction is a few lines:

```
PDA fromGrammar(Grammar g, str s) {
  set[str] nonterminals = g.lhs;
  set[str] terminals = { x | <_, rhs> <- g, x <- rhs, x notin nonterminals };
  return pda("p", {"f"},
      { <"p", "", "", [s, "$"], "q"> }
    + { <"q", "", a, rhs, "q"> | <a, rhs> <- g }
    + { <"q", t, t, [], "q"> | t <- terminals }
    + { <"q", "", "$", [], "f"> });
}
```

The lecture shows the construction for the grammar $I ::= 1 \mid 2 \mid I1 \mid I2$, $E ::= E{+}E \mid E{*}E \mid (E)$ (plus $E ::= I$) in Figure 31. The slide is not consistent about which end of the string is the top: it writes changeTop(I, $I1$) with the top on the left and changeTop(E, $)E($) with the top on the right. The figure uses the first convention throughout.

The slide called "Working automaton" writes the same construction for $S ::= aSb \mid ab$ in Gold, but its first expansion is changeTop(\$, bSa). With the definition of changeTop, that removes the bottom marker \$, so the final pop(\$) can never happen and the automaton recognizes no word. (The test `slideWorkingRejects` checks this for ab and $aabb$.) The construction above keeps \$ at the bottom and puts S on top of it.

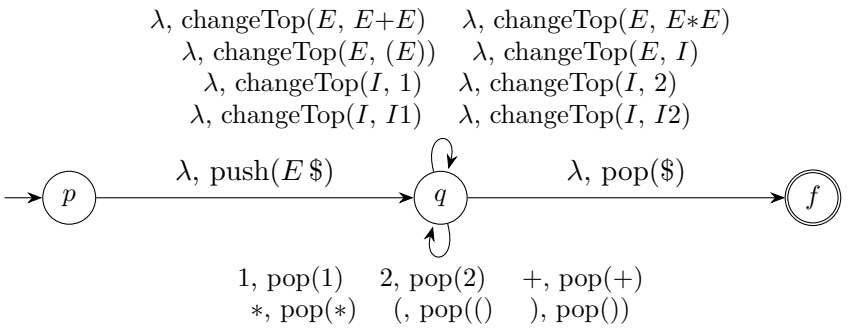


Figure 31: PDA induced by the grammar of I and E.

5.7 Chomsky normal form

Theorem 5.5. *Every context-free language L without λ is generated by a context-free grammar in which every rule has the form $A ::= BC$ or $A ::= a$, with $A, B, C \in N$ and $a \in \Sigma$. Such a grammar is in **Chomsky normal form**.*

The lecture lists the first three steps to get there:

1. eliminate useless symbols, the terminals and nonterminals that are not part of any word of the language;
2. eliminate λ-productions, rules $A ::= \lambda$, by adding to every rule that uses A a copy without it;
3. eliminate unit productions, rules $A ::= B$, by copying the rules of B into A.

Two more steps complete the form: a terminal that appears next to other symbols is replaced by a new nonterminal ($A ::= aB$ becomes $A ::= T_aB$ with $T_a ::= a$), and a long right side is split into pairs ($A ::= BCD$ becomes $A ::= BX$, $X ::= CD$).

For $S ::= aSb \mid ab$ the result is

$$S ::= A_aX \mid A_aB_b \qquad X ::= SB_b \qquad A_a ::= a \qquad B_b ::= b.$$

```
lexical SC = AC XC | AC BC;
lexical XC = SC BC;
lexical AC = "a";
lexical BC = "b";
```

In Chomsky normal form every derivation of a word of length n takes exactly $2n - 1$ steps: $n - 1$ rules of the form $A ::= BC$ and n of the form $A ::= a$. Parsing algorithms such as CYK use this uniform shape (this remark goes beyond the slides).

5.8 A context-free DSL: LDBL

Many programming languages have a context-free structure: markup languages nest elements (`<xml> ... </xml>`), imperative languages nest blocks (`{ ... }`) and functional languages nest expressions in parentheses. The lecture's example is LDBL, a small language to manage the database of an image marketplace. Each entry of the database is an image file. The commands are

- `ADD`, to add an image with its file and its characteristics (attributes and values);
- `REMOVE`, to delete an image given its identifier;
- `SEARCH`, to find images by a list of characteristics combined with logical operators;
- `VIEW`, to see an image;
- assignment of an image variable, which gives a name to an image built from others;
- `ADD IMAGE` and `SUBTRACT IMAGE`, to combine two images;
- `HALT`, to stop.

A statement is the selector of the operation followed by its parameters, each with a name and a value:

```
ADD file image1.img name image properties [[c1, 1], [c2,2]]
```

There are also an `if` with both branches, `if condition then body else body fi`, and a loop with an optional condition, `loop condition do body od`. The slide's grammar is

```
Command   ::= (Operation | Statement)*
Operation ::= Add | Remove | Search | View | Assign | AddImage
            | SubtractImage | Halt | AssignImageVariable
Statement ::= If | Loop
Add       ::= ADD file Name image Name properties [Pairs]
...
If        ::= if Condition then Command else Command fi
Loop      ::= loop [Condition]? do Command od
Name      ::= (a..z)
Pairs     ::= [Name [Value]?, Name|Value]
Condition ::= Name|Value (>,<,==) Name|Value
Value     ::= (0..9)
```

The nesting of `if` and `loop` inside `Command` is what makes the language context-free and not regular. The grammar does not generate its own example, though. `Name ::= (a..z)` is one letter and `Value ::= (0..9)` one digit, while the example has `image1.img` and `c1`; the rule `Add` expects the word `image` where the example has `name`; and `Pairs` has no repetition, so it cannot produce a list of two pairs. A version in Rascal that follows the description and accepts the example:

```
layout Layout = [\ \t\n\r]* !>> [\ \t\n\r];

keyword Reserved = "ADD" | "REMOVE" | "SEARCH" | "VIEW" | "ADDIMAGE" | "SUBTRACTIMAGE
    "
                | "HALT" | "file" | "name" | "properties" | "if" | "then" | "else"
                | "fi" | "loop" | "do" | "od" | "and" | "or";

lexical Name  = ([a-z] [a-z0-9]* !>> [a-z0-9]) \ Reserved;
lexical Value = [0-9]+ !>> [0-9];
lexical File  = Name "." Name;

start syntax Command = (Operation | Statement)*;

syntax Operation
```

```
  = add:      "ADD" "file" File "name" Name "properties" "[" {Pair ","}* "]"
  | remove:   "REMOVE" Name
  | search:   "SEARCH" Referent
  | view:     "VIEW" Name
  | assign:   Name ":=" Image
  | halt:     "HALT"
  ;

syntax Pair     = "[" Name "," Value "]";
syntax Referent = Condition | Referent "and" Condition | Referent "or" Condition;

syntax Image
  = Name
  | "ADDIMAGE" Image Image
  | "SUBTRACTIMAGE" Image Image
  ;

syntax Statement
  = \if:  "if" Condition "then" Command "else" Command "fi"
  | loop: "loop" Condition? "do" Command "od"
  ;

syntax Condition = Atom Comparison Atom;
syntax Atom = Name | Value;
lexical Comparison = "\>" | "\<" | "==";
```

Assignment is written `Name := Image`, where an image is a name or the combination of two images with `ADDIMAGE` or `SUBTRACTIMAGE`. The slide leaves these operations as "...", so their concrete syntax here is a choice.

5.9 Languages that are not context-free

Lemma 5.6 (Pumping lemma for context-free languages)**.** *Let L be an infinite context-free language. There is a constant k, which depends on L, such that every $\omega \in L$ with $|\omega| \geq k$ can be split as $\omega = \alpha\nu\beta\mu\gamma$ with $|\nu\beta\mu| \leq k$ and $\nu\mu \neq \lambda$, and $\alpha\nu^n\beta\mu^n\gamma \in L$ for every $n \geq 0$.*

The idea comes from the parse tree. A long enough word has a path from the root to a leaf where some nonterminal A appears twice. The subtree of the upper A yields $\nu\beta\mu$ and that of the lower A yields β; replacing one by the other, or repeating the upper one, pumps ν and μ together. The game is the same as for regular languages, except that the opponent now splits the word into five parts, in every possible way.

Example: $L = \{a^m b^m c^m \mid m \geq 0\}$ **is not context-free.** Suppose L is context-free and let k be the constant of the lemma. Take $\omega = a^k b^k c^k \in L$. Since $|\nu\beta\mu| \leq k$, the part $\nu\beta\mu$ cannot touch both the a's and the c's: between them there are k b's. There are five cases (Figure 32): $\nu\beta\mu$ lies inside the a's, inside the b's or inside the c's, or it overlaps the boundary between a's and b's, or the boundary between b's and c's. In every case ν and μ together contain at least one symbol, and at most two of the three letters. Take $n = 0$. The word $\alpha\beta\gamma$ has fewer of the letters that appear in $\nu\mu$ and the same number of the letter that does not, so the three counts are no longer equal and

$\alpha\beta\gamma \notin L$. (If ν or μ itself contains two different letters, $n = 2$ also works: the letters come out of order.) This contradicts the lemma, so L is not context-free.

$\underbrace{a\ a\ a}_{\nu\beta\mu}\ a\ a\ b\ b\ b\ b\ b\ c\ c\ c\ c\ c$ case 1: inside the a's

$a\ a\ a\ a\ a\ \underbrace{b\ b\ b}_{\nu\beta\mu}\ b\ b\ c\ c\ c\ c\ c$ case 2: inside the b's

$a\ a\ a\ a\ a\ b\ b\ b\ b\ b\ \underbrace{c\ c\ c}_{\nu\beta\mu}\ c\ c$ case 3: inside the c's

$a\ a\ a\ \underbrace{a\ a\ b\ b}_{\nu\beta\mu}\ b\ b\ b\ c\ c\ c\ c\ c$ case 4: over a's and b's

$a\ a\ a\ a\ a\ b\ b\ b\ \underbrace{b\ b\ c\ c}_{\nu\beta\mu}\ c\ c\ c$ case 5: over b's and c's

Figure 32: Where $\nu\beta\mu$ can fall in $a^k b^k c^k$ (here $k = 5$). It never reaches both the a's and the c's.

The proof on the slide starts with "Suppose L is regular" and repeats the statement of the lemma for regular languages; it also claims that $\nu\beta\mu$ lies inside one block of letters, which leaves out cases 4 and 5 of its own figure. The argument above covers the five cases.

The consequence is that context-free languages cannot count several things at once. A larger class is needed for that: context-sensitive languages, and in general the languages recognized by Turing machines.

5.10 Closure properties

This subsection goes beyond the slides. Context-free languages are closed under union, concatenation and Kleene star: given grammars with initial symbols S_1 and S_2, the rules $S ::= S_1 \mid S_2$, $S ::= S_1 S_2$ and $S ::= S_1 S \mid \lambda$ do it. They are not closed under intersection: $\{a^m b^m c^n\}$ and $\{a^n b^m c^m\}$ are both context-free, and their intersection is $\{a^m b^m c^m\}$. Since $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$, they are not closed under complement either. The intersection of a context-free language with a regular language is context-free: run the PDA and the DFA side by side, as in the product construction.

5.11 Exercises

Exercise 5.7. Write a context-free grammar for well-formed logical formulas (WFF): propositional letters $(p, q, r, s, \dots)$ are WFF; if α is a WFF, so is $\neg\alpha$; if α and β are WFF, so is $\alpha \star \beta$ with $\star \in \{\vee, \Rightarrow, \wedge\}$; if α is a WFF, so is (α). Is the grammar ambiguous?

Solution. The lecture's grammar follows the four cases:

```
syntax WFF    = Prop | Neg | Binary | "(" WFF ")";
syntax Neg    = "¬" WFF;
syntax Binary = WFF Op WFF;
```

```
lexical Op    = "∨" | "∧" | "⇒";
lexical Prop  = [p-s];
```

It is ambiguous: $p \wedge q \vee r$ has a tree with $p \wedge q$ as the left operand of $\vee$ and another with $q \vee r$ as the right operand of $\wedge$. To remove the ambiguity, give the operators levels as in the expression grammar (for example $\neg$, then $\wedge$, then $\vee$, then $\Rightarrow$), or require parentheses around every binary formula.

Exercise 5.8. Write a pushdown automaton for the language of WFF. Use the symbols of the lecture's Gold code: p to s for letters, `N` for $\neg$ and `A`, `O`, `I` for $\wedge$, $\vee$, $\Rightarrow$.

Solution. Two states are enough (Figure 33). In E the automaton expects a formula: a letter completes one, `N` expects another, and `(` pushes a marker and expects the formula inside. In F a formula has just been completed: an operator expects the right operand, and `)` closes a parenthesis by popping its marker. F is final, so a word is recognized when it ends after a complete formula with all parentheses closed.

```
PDA wff = pda("E", {"F"}, {
  *{ <"E", l, "", [], "F"> | l <- ["p", "q", "r", "s"] },
  <"E", "N", "", [], "E">,
  <"E", "(", "", ["P"], "E">,
  *{ <"F", op, "", [], "E"> | op <- ["A", "O", "I"] },
  <"F", ")", "P", [], "F">
});
```

The tests compare this PDA with a direct recognizer of the grammar on every word of length up to 5 over $\{p, \mathtt{N}, \mathtt{A}, (,)\}$.

The lecture's automaton has four states, and D, the state reached after an operator, is final. It accepts `A` (the operator alone) and `pA` ($p\wedge$ with no right operand), which are not formulas. It also handles only the letter p and the operator `A`.

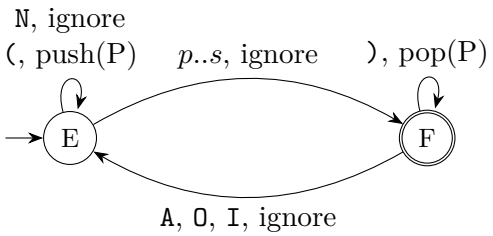


Figure 33: PDA for well-formed formulas.

Exercise 5.9. Write a pushdown automaton for the expressions of the grammar `Exp ::= Exp + Term | Term`, `Term ::= Term * Fact | Fact`, `Fact ::= ( Exp ) | (0..9)+`.

Solution. Apply the construction from grammars: one state q with a transition λ, changeTop(A, β) for each rule $A ::= \beta$ and a transition a, pop(a) for each terminal. With numbers written as the single digit 1:

```
Grammar exp = {
  <"Exp",  ["Exp", "+", "Term"]>,  <"Exp",  ["Term"]>,
  <"Term", ["Term", "*", "Fact"]>, <"Term", ["Fact"]>,
```

```
  <"Fact", ["(", "Exp", ")"]>,     <"Fact", ["1"]>
};
```

`fromGrammar(exp, "Exp")` is the automaton. The left recursion is not a problem for the PDA because it is nondeterministic, but a deterministic top-down parser would loop on `Exp ::= Exp + Term`, which is why the next block removes it.

Exercise 5.10. Which words of length at most 4 does $\mathcal{G}_{2'}$ generate? Give a parse tree for *baab*.

Solution. The words with as many a's as b's: λ, ab, ba, $aabb$, $abab$, $abba$, $baab$, $baba$, $bbaa$. For $baab$: $S \Rightarrow bA \Rightarrow baS \Rightarrow baaB \Rightarrow baabS \Rightarrow baab$, so the tree is $S(b, A(a, S(a, B(b, S(\lambda)))))$.

Exercise 5.11. Fix the Grace grammar so that it generates the method of the lecture, including the assignment `x := x - 1`.

Solution. Make the second line a rule of `Exp`, `Exp ::= Ident | Num | print(Exp)`, and add `Stmt ::= Ident := Exp`. To generate the final call `run`, add a rule for programs, `Program ::= S \n Ident`, and make `Program` the initial symbol.

Exercise 5.12. Use the pumping lemma to show that $\{a^m b^n c^m d^n \mid m, n \geq 0\}$ is not context-free.

Solution. Take $\omega = a^k b^k c^k d^k$. Since $|\nu\beta\mu| \leq k$, the part $\nu\beta\mu$ touches at most two adjacent blocks. If it touches the a's (or the c's) but not the c's (or the a's), pumping with $n = 0$ changes the number of a's without changing the number of c's, or the other way around. The same holds for b's and d's. In every case the pumped word is not in the language.

References

[1] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation.* Pearson Addison-Wesley, 3rd edition, 2007.

[2] Kent D. Lee. *Foundations of Programming Languages.* Springer, 2nd edition, 2017.

[3] Bruce J. MacLennan. *Principles of Programming Languages: Design, Evaluation, and Implementation.* Oxford University Press, 3rd edition, 1999.

[4] Terrence W. Pratt and Marvin V. Zelkowitz. *Programming Languages: Design and Implementation.* Prentice Hall, 4th edition, 2001.